Awarded by

AI-Based Driver Safety and Assistance System

A thesis submitted

by

Mohamed Mostafa Mohamed El-Basyouni (Team Leader)
Michael Magdy Amin Sidhom
Ali Ibrahim Ahmed Othman
Karim Mustafa Ali Ibrahim
Mohamed Osama Bahnasy Abdel Halim

Under supervision of

General Prof. Dr. Kamel Elhadad
Military Technical College

A dissertation submitted in partial fulfillment of the requirements for the degree of Digilians 9 Months Diploma in Applied AI and Data Analytics

(Cairo 2026)

Awarded by

Approval Sheet
AI-Based Driver Safety and Assistance System

A thesis submitted
by

Mohamed Mostafa Mohamed El-Basyouni (Team Leader)
Michael Magdy Amin Sidhom
Ali Ibrahim Ahmed Othman
Karim Mustafa Ali Ibrahim
Mohamed Osama Bahnasy Abdel Halim

A dissertation submitted in partial fulfillment of the requirements for the degree of Digilians 9 Months Diploma

Applied AI and Data Analytics Track

Approved by the examination committee

Name	Signature
Prof.	
Prof.	
Prof.	

Abstract

Driver fatigue is a persistent contributor to road traffic collisions, and its early behavioural indicators - prolonged eye closure and yawning - are observable from a single in-cabin camera. This project develops a complete vision-based driver monitoring system, from dataset construction through model selection to browser-based deployment.

The work began from a 57,098-image corpus assembled from several independently annotated sources. Analysis established that the corpus was a merge of separate single-task datasets - an eye-state corpus, a yawning corpus, and driver-monitoring session recordings - combined into one three-class label space without re-annotation. This produced systematic, source-correlated missing supervision rather than random label noise. A structured reconciliation process, including human verification of sampled images against spatial evidence gates, produced a source-aware supervision manifest recording which classes each source family can be trusted to supervise. Group-aware splitting was applied so that near-duplicate frames from the same recording session could not span the training and test partitions, and the resulting split was independently verified for leakage. The final dataset contains 50,654 images and 68,292 annotated instances across the classes closed_eye, open_eye and yawning.

Twenty-one training runs were carried out across four detection architecture families - YOLO26, YOLOv11, the transformer-based RF-DETR, and a simplified Faster R-CNN implemented from scratch - spanning 950 epochs and 277.57 GPU-hours, at input resolutions from 384 to 960 pixels and under two augmentation intensities. All models were evaluated under a single protocol on a held-out 5,589-image test partition, reporting both standard and label-gap-corrected metrics, the latter accounting for the partial annotation identified during dataset analysis.

Two YOLO26n configurations were selected for deployment, at 480 and 960 pixel input, achieving 82.72 % and 82.75 % corrected mAP@0.5 respectively. Both were exported to ONNX with non-maximum suppression compiled into the graph and converted to half precision, halving model size with no measurable loss of detection agreement. The models execute entirely on the user's own device through a browser application using WebGPU with a multi-threaded WebAssembly fallback, so that video is never transmitted to any server.

Temporal interpretation is performed by rolling-window PERCLOS aggregation with a debounced state machine. Learned temporal modelling using recurrent or sequence architectures was not implemented in this work and is identified as the next stage of development.

Acknowledgments

We would like to express our deepest gratitude and sincere appreciation to our supervisor, **Prof. Dr. Kamel Elhadad**, for his invaluable guidance, continuous support, and constructive technical direction throughout the duration of this project. His extensive expertise, critical feedback, and rigorous academic mentorship played an instrumental role in shaping our engineering methodology, refining our analytical decisions, and maintaining a high standard of scientific rigor in our findings.

We also extend our sincere appreciation to the **Military Technical College**, specifically the **Department of Computer Engineering and Artificial Intelligence**, for providing an exceptionally supportive academic environment. We are profoundly grateful for the access to the advanced laboratory facilities, computing infrastructure, and experimental resources, without which the implementation, validation, and empirical analyses presented in this work could not have been achieved.

Furthermore, we gratefully acknowledge the **Digilians (Digital Pioneers) Initiative** for offering the institutional platform, comprehensive training, and collaborative ecosystem under which this research was conducted. The opportunities, practical exposure, and professional development afforded by the initiative served as an essential catalyst for the successful realization and completion of this endeavor.

Table of Contents

List of Figures

List of Tables

Nomenclature

Abbreviations

ADAS	Advanced Driver Assistance System
AMP	Automatic Mixed Precision
AP	Average Precision
CNN	Convolutional Neural Network
COCO	Common Objects in Context
DETR	Detection Transformer
DFL	Distribution Focal Loss
FP16	Half-Precision Floating Point
FP32	Single-Precision Floating Point
FPS	Frames Per Second
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	mean Average Precision
NMS	Non-Maximum Suppression
ONNX	Open Neural Network Exchange
PERCLOS	Percentage of Eye Closure
RoI	Region of Interest
RPN	Region Proposal Network
SAOD	Sparsely Annotated Object Detection
VRAM	Video Random Access Memory
WASM	Web Assembly
YOLO	You-Only-Look-Once

Chapter (1)

Introduction

1.1 Overview

Driver fatigue is a recognised and persistent contributor to road traffic collisions. Unlike mechanical failure, it develops gradually and is frequently unrecognised by the driver until control is already degraded. Its value as a target for automated detection comes from the fact that it is expressed physically before it becomes catastrophic: the eyes close for longer than a blink, the rate of yawning rises, and both are visible to a camera positioned inside the vehicle cabin.

This project develops a driver monitoring system that observes those cues in real time using computer vision. A detection model locates and classifies three visual states - a closed eye, an open eye, and a yawning mouth - in each video frame. A temporal layer converts that per-frame evidence into a fatigue assessment, because a single frame showing closed eyes is indistinguishable from an ordinary blink; only duration separates the two.

The system is designed to run entirely on the user's own device through a web browser. This is a deliberate architectural constraint rather than a convenience: a driver monitoring system processes continuous video of a person's face, and transmitting that video to a remote server introduces both a privacy exposure and a latency dependency on network conditions that a safety system should not carry.

1.2 Problem Statement

Building such a system presents three coupled problems.

The first is data. Publicly available driver-monitoring datasets are typically annotated for a single task - either eye state or yawning, rarely both - and are frequently derived from video, which means consecutive frames are near-duplicates of one another. Assembling a corpus large enough to train a detector therefore requires merging sources, and merging sources annotated for different tasks introduces a form of label incompleteness that is systematic rather than random.

The second is model selection under deployment constraints. A model intended to run in a browser on unknown consumer hardware is constrained not only by accuracy but by download size, memory footprint, and inference latency. The most accurate model available is not necessarily the correct choice.

The third is evaluation integrity. When a dataset contains systematic missing annotation, a correct detection of an unlabelled object is scored as an error. Any accuracy figure computed without

accounting for this is misleading, and comparisons between models built on such figures are unreliable.

1.3 Objectives

The objectives of this project are:

1. To construct a leakage-free, three-class object detection dataset for driver drowsiness from heterogeneous merged sources, and to characterise and account for the annotation incompleteness that merging introduces.

2. To train and comparatively evaluate detection architectures spanning single-stage convolutional, transformer-based, and two-stage designs, under a single consistent evaluation protocol.

3. To quantify the accuracy cost of reduced input resolution, which is the principal lever available for deployment on constrained devices.

4. To select and export models suitable for real-time in-browser inference, and to verify that the export and precision-reduction steps do not degrade detection behaviour.

5. To implement a temporal interpretation layer converting per-frame detections into a fatigue state.

1.4 Scope

This project covers dataset construction and auditing, model training and comparative evaluation, model export and precision conversion, latency measurement, and browser-based deployment with temporal fatigue estimation.

It does not cover physiological sensing of any kind - electroencephalography, heart-rate monitoring, or steering-input analysis - all of which appear in the wider driver-monitoring literature but fall outside a camera-only approach. It does not cover in-vehicle hardware integration. Learned temporal modelling using recurrent or sequence architectures is identified as future work and was not implemented.

1.5 Research Questions

The work is organised around four questions:

RQ1. How can a usable detection dataset be constructed from merged single-task sources, and what is the measurable effect of the resulting annotation incompleteness on reported accuracy?

RQ2. Which detection architecture family offers the best accuracy for this task, and does architectural novelty translate into measurable advantage?

RQ3. What accuracy is lost by reducing input resolution, and does that loss justify the deployment benefit it is assumed to provide?

RQ4. Does half-precision conversion, applied to reduce model download size, measurably change detection behaviour?

1.6 Solution Approach

The approach proceeds in four stages. The corpus is first audited and reconstructed, producing a source-aware record of which classes each contributing source can be trusted to supervise, together with a group-aware partition that prevents near-duplicate frames from spanning the train and test boundary.

Twenty-one training runs are then carried out across four architecture families and four input resolutions, and all are evaluated under one protocol on the same held-out partition, with both standard and annotation-corrected metrics reported.

Two configurations are selected for deployment on the basis of accuracy, size and measured latency together. These are exported to ONNX with suppression compiled into the graph, converted to half precision, and verified against their full-precision counterparts on real test images before release.

The exported models are executed in a browser through a dedicated worker thread, and their per-frame output is aggregated by a rolling-window fatigue estimator with debounced state transitions.

1.7 Document Organisation

Chapter (2) reviews related work in vision-based driver monitoring. Chapter (3) describes the dataset, its construction, the architectures evaluated, and the training and evaluation protocols. Chapter (4) presents the experimental results, the measured performance characteristics, and the deployed system. Chapter (5) states the conclusions and identifies future work.

Chapter (2)

Literature Review

2.1 Vision-Based Driver Monitoring

Vision-based driver monitoring rests on the premise that fatigue produces observable facial behaviour before it produces loss of vehicle control. George and Rochit [8] survey the ADAS drowsiness-detection landscape and identify the recurring obstacles to reliable deployment: algorithmic limitations, sensor reliability, real-time processing constraints, human-machine interface design, and the complex interplay of physiological and environmental factors that complicates accurate detection.

Two broad strategies appear in the literature. The first extracts geometric measurements from detected facial landmarks and applies thresholds to them. Arava and Sundaram [2] follow this approach, combining a lightweight YOLOv5s detector with 3D facial keypoints and computing eye and mouth aspect ratios against threshold values indicative of closure and yawning. The second treats the visual states themselves as detection targets, so that the network learns the appearance of a closed eye directly rather than inferring it from landmark geometry.

The present project follows the second strategy. Landmark-based geometric ratios depend on reliable landmark localisation, which degrades precisely under the conditions - poor illumination, occlusion, extreme pose - where a fatigue detector is most needed. Treating eye and mouth state as detection classes removes that dependency.

2.2 Detection Architectures for Driver Monitoring

The YOLO family dominates the applied driver-monitoring literature, and successive generations have been compared directly. Herath et al. [5] fine-tune seven YOLO variants spanning v5 through v11 on the UTA-RLDD dataset and report that while YOLOv9c achieved the highest accuracy in their study, YOLOv11n offered the best balance between precision and inference efficiency, which they identify as making it the more suitable choice for embedded deployment.

That distinction - highest accuracy versus best deployable balance - is the same one that governs model selection in this project, and it recurs throughout the applied literature. Chen et al. [3] pursue it explicitly, proposing MG-YOLOv8 as a deliberately lightweight real-time fatigue detector combining multiple facial features, on the premise that a model which cannot meet a latency budget on target hardware is not useful regardless of its accuracy.

Go et al. [4] address a component that most drowsiness pipelines treat as solved. They benchmark seven face detectors, including YOLOv11n, SSD MobileNet, YuNet and classical Haar cascades, and make a methodological observation directly relevant here: many prior studies use detector-generated outputs as ground truth, which introduces bias and can inflate reported performance. Their concern is that supervision quality itself is frequently taken for granted.

Beyond single-frame detection, Yusuf et al. [7] fuse multi-level features with a long short-term recurrent network, treating drowsiness as a temporal pattern rather than a per-frame property. This is the direction identified as future work in Chapter (5) of the present document.

2.3 Data Quality and Annotation

The most directly relevant prior work concerns annotation quality rather than architecture. Mujtaba et al. [1] observe that existing yawning datasets carry systematic noise arising from coarse temporal annotation: labels are applied at the video-segment level, so frames within a segment labelled as containing a yawn may not themselves show one. They address this by developing a semi-automated labelling pipeline with human-in-the-loop verification, producing frame-level annotations for the YawDD videos, and demonstrate that training on the refined labels improves model quality on edge platforms.

Their finding - that a substantial performance limitation originated in the labels rather than in the model - directly parallels the central finding of the present project. The mechanism differs: Mujtaba et al. address temporal coarseness within a single dataset, whereas the corpus used here suffers from class-wise incompleteness caused by merging datasets annotated for different tasks. The methodological response is comparable in both cases: human verification of a sampled subset, used to establish a systematic rule rather than to correct individual images by hand.

Alzami et al. [6] approach data limitation from the opposite direction, treating it as an image-quality problem. They apply Bayesian optimisation to select contrast-enhancement parameters adaptively for low-light frames, using a perceptual image-quality measure as the optimisation objective, and report significant improvement over both unprocessed frames and fixed-parameter enhancement on the NITYMED dataset. Their approach is not adopted here - the present system relies on photometric augmentation during training rather than on inference-time preprocessing - but it illustrates an alternative response to the same underlying difficulty.

2.4 Research Gap

Three observations follow from the reviewed work.

First, comparative studies of detection architectures for driver monitoring are typically conducted within a single architectural lineage. Herath et al. [5] compare seven YOLO variants; Go et al. [4] compare face detectors. Comparison across genuinely different detection paradigms - single-stage convolutional, transformer-based set prediction, and two-stage proposal-and-refine - on one dataset under one protocol is less common.

Second, dataset quality is recognised as a limitation [1], [4], but the specific problem of merging datasets annotated for different tasks into a shared label space, and the systematic class-wise missing supervision this produces, is not addressed in the reviewed set. Nor is its effect on evaluation, as distinct from its effect on training.

Third, deployment is consistently framed in terms of embedded and edge hardware [1], [3], [5]. In-browser inference, where the model must be downloaded over a network and executed inside a sandboxed runtime on unknown consumer hardware, imposes a different constraint set - download size becomes a primary cost - and is not represented in the reviewed work.

This project addresses all three: it compares four architecture families under a single protocol, it characterises and corrects for merge-induced annotation incompleteness in both training and evaluation, and it targets browser deployment directly.

Coverage note. The reviewed set comprises the eight works available to this study. It does not include primary references for several components the project uses: the RF-DETR architecture, the original definition and validation of PERCLOS, the sparsely-annotated object detection literature, or the Faster R-CNN design on which the from-scratch implementation is based. These components are described from their own technical documentation and source implementations in Chapter (3) rather than attributed to unrelated citations.

Chapter (3)

Methodology

3.1 Data Collection

This chapter describes the data, the preparation pipeline, the detection architectures evaluated, and the evaluation protocol. Every quantitative statement is drawn from the project's own artefacts, and the provenance of each is stated where it is not self-evident.

3.1.1 Task Formulation

Driver drowsiness is expressed through observable facial behaviour: the eyes close for abnormally long intervals, and the mouth opens in yawns. A system intended to run inside a vehicle must recognise these cues from a single camera view, in real time, under uncontrolled illumination.

The task was framed as object detection over three classes - closed_eye, open_eye and yawning - rather than as whole-image classification. This choice has consequences throughout the project. A classifier assigns one label to an entire frame and cannot express that one eye is closed while the other is open, nor localise which region of the face produced the evidence. A detector returns localised, independently scored instances, which is what a temporal fatigue estimator requires: the proportion of time the eyes are closed cannot be computed from a frame-level label alone.

3.1.2 Corpus Composition

The working corpus was assembled from 57,098 images drawn from several independently annotated sources: public dataset exports, stock photography, compiled drowsy-driving photograph sets, and driver-monitoring session recordings.

The decisive property of this corpus, and the origin of most of the methodological work in this chapter, is that it is not a single dataset. It is a merge of separate single-task datasets - an eye-state corpus annotated only for eye classes, a yawning corpus annotated only for mouth state, and several session recordings - combined into one three-class label space without re-annotation.

The consequence is that many images contain objects that are genuinely present but were never labelled, because the source that contributed the image was never concerned with that class. An image from the eye-state corpus may show a yawning driver but carry no yawning box. This is not random annotation noise. It is systematic, source-correlated missing supervision, and it invalidates the standard assumption that an unlabelled region is background. In the detection literature this setting is

termed sparsely annotated object detection, and it is known to degrade training by supplying false-negative supervision: unlabelled true objects are scored as background, teaching the detector to suppress exactly the evidence it should learn.

After the cleaning and de-duplication pipeline described in Section 3.2.1, the corpus was reduced to a final set of 50,654 images containing 68,292 annotated instances.

Before assembling the corpus described above, the eight studies reviewed in Chapter 2 were examined for a directly reusable, ready-made alternative. None supplied a corpus that could be adopted as-is for a three-class (closed_eye, open_eye, yawning) detection task built from multiple independent sources. The table below summarises what dataset each study actually used and how accessible that dataset is.

Dataset accessibility across the studies reviewed in Chapter 2

No.	Study	Dataset(s) used	Accessibility of that dataset
[1]	Mujtaba et al. (2025)	YawDD, re-annotated at frame level ("YawDD+")	Re-annotations released online at a public repository; but single-task (yawning only) frame labels layered on one source video set — not a ready multi-class eye/mouth corpus.
[2]	Arava & Sundaram (2024)	YawDD + CEW + DrivFace + DROZY (compiled)	Only the four source datasets are individually linked; the authors' own compiled and labelled 8,021-image working set has no separate release.
[3]	Chen et al. (2025)	WIDER FACE + YawDD (benchmarks) + 12-participant in-house data	The two benchmarks are public; the paper's own supplementary participant data is not released ("further inquiries can be directed to the corresponding author").
[4]	Go et al. (2025)	NITYMED + author-annotated ground truth (~1,229 frames)	No availability statement is given for either the source videos or the authors' own frame-level annotations.
[5]	Herath et al. (2025)	UTA-RLDD	Called "publicly available" in the paper; the source dataset's own distribution has historically required a request/consent process with the originating researchers, since it is identifiable face video.
[6]	Alzami et al. (2026)	NITYMED	No availability statement is given in the paper.
[7]	Yusuf et al. (2024)	NTHU-DDD	Explicitly obtained "with the permission of" the source laboratory — permission-gated, not an open download.
[8]	George & Rochit (2025)	— (literature survey; proposes no dataset of its own)	Not applicable.

As the table shows, even where a source dataset is nominally public, none combines multi-source coverage, three-class detection-ready supervision, and unrestricted access in the way this project required: some releases are gated behind a permission request to the originating laboratory, some named “public” datasets carry access conditions the citing paper does not itself resolve, and in several cases the paper’s own working dataset is a private compilation or annotation layered on top of the cited public source, with no separate release of that compiled or annotated version. Building a merged, custom-annotated corpus was therefore the only path available to reach three-class, multi-source, detection-ready supervision at this project’s scale — the same position several of these studies were also in, which is why their own compilation or annotation effort was necessary in the first place.

The custom corpus assembled for this project — all 57,098 source images, the merged and re-mapped three-class annotations, and the reproducible train/validation/test split — is itself made publicly available at: https://drive.google.com/drive/folders/126mrDWhsI_PmlOLjTomMWSjUjySS6XxT?usp=drive_link.

3.1.3 Annotation Protocol

Annotations use the standard YOLO detection format: one text file per image, one line per instance, giving the class index and the normalised centre coordinates, width and height of the bounding box.

Because the corpus was inherited rather than annotated from scratch, the annotation work in this project was verification and reconciliation rather than initial labelling. Two automated attempts to infer annotation completeness were made and both were rejected as unreliable. The problem was resolved through structured human review, in which sampled images were examined against a set of spatial evidence gates before any conclusion about missing labels was accepted. The decisive gate required that labelled geometry be cross-validated against an independent facial-geometry prediction: an eye-only image was accepted as genuinely eye-only supervision only if its labelled boxes fell inside the predicted eye region of the detected face.

The outcome is a source-aware supervision manifest recording, for each source family, which classes it can be trusted to supervise. This manifest is what makes the corrected evaluation in Section 3.6.2 possible, and it is the project's principal methodological contribution.

3.2 Data Preparation

3.2.1 Corpus Construction and Quality Control

The path from the 57,098-image raw corpus to the final 50,654-image dataset consisted of eight stages, each independently verified: ingestion and integrity scanning; geometric and bounding-box scale analysis to detect malformed annotations; the annotation completeness investigation described above; construction of the source-aware supervision manifest; repair of malformed annotation geometry; group-aware splitting; independent leakage verification run against the finished splits rather than against the splitting code; and final audit and export to Ultralytics YOLO format.

The raw corpus was treated as read-only throughout. No stage modified a source file in place.

3.2.2 Data Augmentation

Augmentation was applied online during training. Two intensity levels were used across the experimental programme, both recorded in each run's own configuration file.

Table (3.1) Augmentation parameters by intensity level

Parameter	Moderate	Worst-case
fliplr	0.5	0.5
flipud	0.0	0.0
degrees	15.0	25.0
shear	4.0	8.0
perspective	0.0005	0.0008
translate	0.15	0.25
scale	0.6	0.7
hsv_h	0.02	0.03
hsv_s	0.7	0.8
hsv_v	0.5	0.6
erasing	0.3	0.40
auto_augment	randaugment	randaugment
mosaic / mixup / cutmix / copy_paste	0.0 (disabled)	0.0 (disabled)

The HSV, rotation and erasing settings simulate real in-cabin failure modes: exposure swings between daylight and dashboard lighting, head rotation, and partial occlusion of the face by hands, spectacles or seat-belt hardware.

The compositing augmentations - mosaic, mixup, cutmix and copy_paste - were disabled in every run at both intensity levels. This is a structural decision rather than a tuning choice. Each of these operations builds a training image by combining regions from several source images. In a corpus where supervision trustworthiness is a property of the source family, compositing destroys exactly that property: a mosaic tile assembled from an eye-only image and a yawn-only image produces a composite whose correct supervision mask is undefined. Compositing and source-aware label handling are therefore mutually exclusive, and source awareness was the more valuable of the two.

3.2.3 Splitting and Class Distribution

Splitting was group-aware. Because the corpus includes video-derived session recordings, adjacent frames from one session are near-duplicates. A naive random split would place near-identical frames on both sides of the train and test boundary, producing a test score that measures memorisation rather than generalisation. Frames belonging to the same session were therefore constrained to the same split, and the resulting partition was verified for leakage by a separate, independent procedure.

Table (3.2) Final dataset composition

Split	Images	Instances	closed_eye	open_eye	yawning
Train	39,627	53,620	19,366	17,657	16,597
Validation	5,438	7,245	2,910	2,002	2,333
Test	5,589	7,427	2,395	2,327	2,705
Total	50,654	68,292	**24,671 (36.1 %)**	**21,986 (32.2 %)**	**21,635 (31.7 %)**

The class distribution is close to uniform: the largest class accounts for 36.1 % of instances and the smallest for 31.7 %. No class-rebalancing procedure was therefore required or applied, since a spread of 4.4 percentage points does not constitute the imbalance that would justify resampling or loss reweighting.

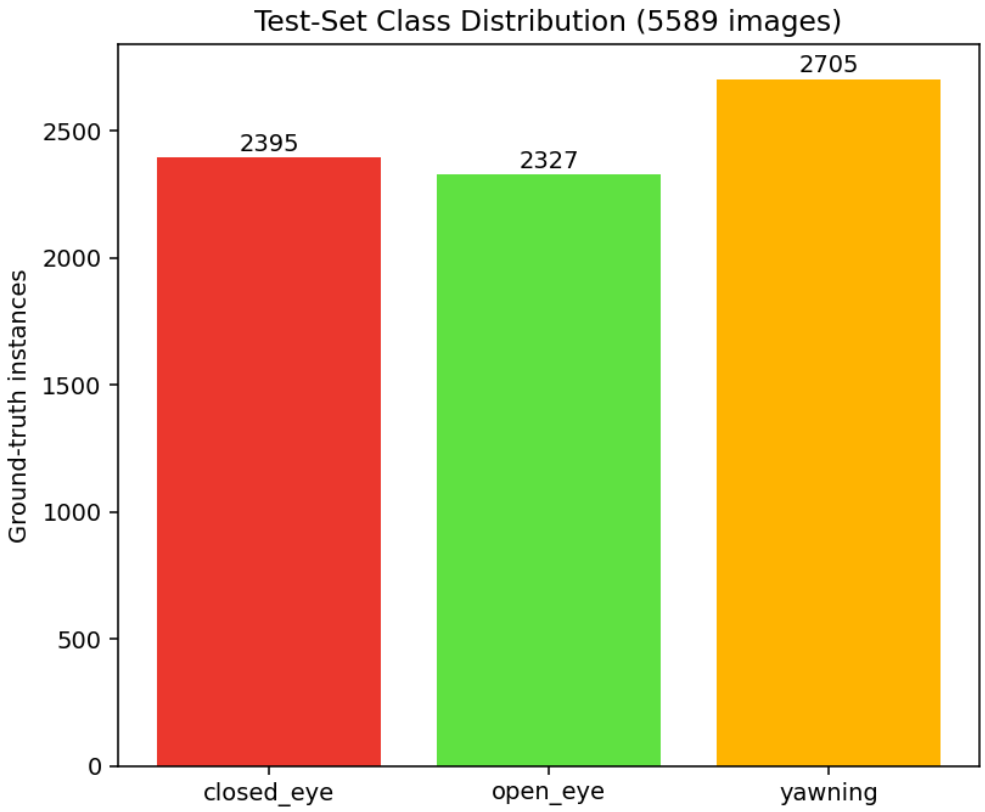

Fig. (3.1) Class distribution of the held-out test partition

3.3 Temporal Fatigue Estimation

Per-frame detection alone does not constitute drowsiness detection. A single frame in which the eyes are closed is indistinguishable from an ordinary blink; what separates a blink from a microsleep is duration. The detector's per-frame output is therefore aggregated over time before any driver-state decision is made.

3.3.1 PERCLOS

The aggregation measure is PERCLOS - the proportion of a time window during which the eyes are closed. PERCLOS was established as the fraction of a one-minute interval in which the eyelid covers more than 80 % of the pupil, and has been repeatedly validated as among the most reliable behavioural indicators of alertness.

In this project PERCLOS is computed over a rolling window of recent frames from the ratio of closed_eye to open_eye detections, rather than from eyelid-aperture measurement, because the detector's output is discrete class evidence rather than a continuous aperture value.

3.3.2 Driver-State Decision

A debounced state machine converts the rolling PERCLOS value and yawn frequency into a discrete driver state with escalating alert levels. Debouncing is required because raw per-frame detection is noisy: a single misdetection must not be able to trigger an alarm, and a genuine microsleep must not be cancelled by a single recovered frame.

The temporal reasoning implemented in this project is rolling-window aggregation with debounced state transitions. Learned temporal modelling - recurrent or sequence architectures trained on temporal data - was not implemented and is identified as future work in Chapter (5). This distinction is maintained deliberately throughout this document.

3.4 Detection Architectures

Four architecture families were evaluated, spanning the meaningful axes of the design space: single-stage convolutional, transformer-based set prediction, and two-stage proposal and refinement. Within the single-stage families two capacities each were trained, so that the accuracy contribution of model size could be separated from that of architecture generation.

Table (3.3) Detection architectures evaluated

Family	Capacity	Parameters (M)	Paradigm
YOLO26	n / s	2.50 / 9.95	Single-stage convolutional
YOLOv11	n / m	2.59 / 20.05	Single-stage convolutional
RF-DETR	Nano / Small	30.15 / 32.02	Transformer set prediction
Faster R-CNN	From scratch	-	Two-stage proposal and refine

3.4.1 YOLO26

YOLO26 is the most recent single-stage family evaluated and provided both models selected for deployment. Two capacities were trained: YOLO26n at 2.50 M parameters and YOLO26s at 9.95 M.

The property that made this family decisive for deployment is that its export path produces an ONNX graph with non-maximum suppression compiled into the graph itself. The exported model emits a fixed tensor of 300 already-suppressed detections, each row giving box coordinates, confidence and class index. For browser deployment this removes an entire post-processing stage from the client, which would otherwise run suppression in JavaScript on every frame.

3.4.2 YOLOv11

Two YOLOv11 capacities were trained as comparison points: YOLO11n at 2.59 M parameters and YOLO11m at 20.05 M. The YOLOv11 family is well represented in the driver-monitoring literature, which makes it a meaningful reference against which a newer architecture can be judged.

3.4.3 RF-DETR

RF-DETR is a real-time detection transformer built on a vision-transformer backbone, representing the DETR architectural family rather than the single-stage convolutional lineage. Two capacities were evaluated: RF-DETR-Nano at 30.15 M parameters and RF-DETR-Small at 32.02 M.

Transformer detectors dispense with hand-designed anchors and with non-maximum suppression, instead predicting a fixed set of object queries matched to ground truth during training. Their inclusion tests whether that architectural difference translates into measurable advantage on this task.

3.4.4 Simplified Faster R-CNN Implemented From Scratch

A two-stage detector was additionally implemented from first principles, without a detection framework, as an educational and comparative exercise. Its design follows the Faster R-CNN pattern: a custom convolutional backbone of four blocks at stride 16 produces a feature map from a 640-pixel input; a region proposal network over 14,400 anchors predicts objectness and box offsets; decoding and suppression yield approximately 1,000 proposals; RoI alignment extracts fixed-size features per proposal; and a fully-connected head produces class scores and box refinements.

The model is trained against four simultaneous losses - region proposal objectness, region proposal box regression, detection classification, and detection box regression - over 50 epochs, and classifies three foreground classes plus an explicit background class. A refined configuration extends this to 65 epochs, resuming from the tuned checkpoint after a further pass completed training labels the model itself had flagged as missing (Section 4.8).

This model was developed on a separate track and evaluated on a test partition of 5,705 images, whereas all other architectures in this study were evaluated on the 5,589-image partition defined in Section 3.2.3. Its results are therefore reported separately in Chapter (4) and are not entered into the comparative ranking, because the two figures are not measured on the same data.

3.5 Training Protocol

All training and evaluation was performed on a single NVIDIA RTX 2000 Ada Generation GPU with 17.18 GB of VRAM, under Windows, using Python 3.11.15, PyTorch 2.6.0 with CUDA 12.4, and Ultralytics 8.4.64.

Table (3.4) Training hyperparameters (deployed 960-pixel configuration)

Parameter	Value
Optimiser	AdamW
Initial learning rate (lr0)	0.001
Final learning-rate factor (lrf)	0.01
Schedule	Cosine
Momentum	0.9
Weight decay	0.0005
Warm-up epochs	3.0
Mixed precision	Enabled
Box / class / DFL loss weights	7.5 / 1.5 / 1.5
Batch size	32
Early-stopping patience	12

Input resolutions of 384, 480, 640 and 960 pixels were used across the programme, allowing the accuracy cost of reduced input size - the principal lever available for deployment on constrained hardware - to be measured directly rather than assumed.

One implementation constraint applied throughout. The number of data-loading worker processes was held at two for most runs; higher counts caused repeated data-loader failures on the training platform, and two was the only value verified stable across every batch size used.

3.6 Evaluation Protocol

3.6.1 Metrics

All models were evaluated on the held-out test split of 5,589 images containing 7,427 ground-truth instances, at an IoU threshold of 0.5 and an operating confidence of 0.35.

The primary metric is mAP@0.5, the mean over classes of average precision at IoU 0.5. Average precision integrates over the full precision-recall curve and is therefore independent of the operating confidence threshold. Precision, recall, F1 and the confusion matrix are threshold-dependent and are reported at the 0.35 operating point.

Per-class average precision is reported alongside the mean throughout, because the three classes are not equally important to the application: closed_eye is the class from which microsleep evidence is derived, and a mean that concealed a regression on that class would be misleading.

3.6.2 Label-Gap-Corrected Evaluation

The systematic missing supervision described in Section 3.1.2 distorts evaluation as well as training. When a model correctly detects a yawning driver in an image drawn from a source family that never annotated yawns, the standard protocol scores that correct detection as a false positive, because no matching ground-truth box exists. Precision is therefore understated by an amount that depends on which source families are present in the test set.

A corrected metric is reported alongside the raw one. Under the correction, a prediction of a class that was never annotated anywhere in the source family that contributed the image is ignored - counted as neither a true nor a false positive. This is the standard convention for partially annotated data, equivalent to the iscrowd mechanism in the COCO protocol.

Two properties of this correction matter. First, ground truth is never modified, so recall and false-negative counts are identical between the raw and corrected figures by construction; only precision, and the average precision derived from it, can differ. Second, the correction is scoped to the source family rather than to the individual image: a class is ignored on an image only if the entire family contributing that image contains no annotation of that class anywhere, and a minimum family size is required before the rule is applied at all, so that a small family cannot be misclassified as blind to a class by chance.

On the test split this affects eye predictions on 1,039 images and yawn predictions on 699 images. The resulting difference is small - typically under half a percentage point of mAP@0.5 - but it is reported explicitly rather than silently, and both figures appear in Chapter (4).

3.6.3 Evidence Integrity

Every evaluation reported in this document was produced by a single evaluation implementation, run over the same test partition, for every model. This uniformity was adopted deliberately after inherited evaluation records from an earlier phase of work were found to be unreliable: several chart images distributed across different models' result folders were byte-identical to one another, including a single confusion-matrix image shared across eleven runs spanning five different architectures with reported accuracies ranging from 66 % to 92 %. Such an image cannot represent all of those models.

All results were therefore re-measured from the model checkpoints themselves, and the regenerated chart set was verified by content hashing to confirm that every model's figures are distinct. Where a required artefact genuinely could not be reconstructed - training histories for six runs whose per-epoch logs were not preserved - the absence is recorded explicitly in the results archive rather than filled with a substitute.

3.7 Deployment Pipeline

The trained detector is deployed as an in-browser application, so that video never leaves the user's device. The pipeline exports the trained checkpoint to ONNX at the model's own training resolution with suppression compiled into the graph and the graph simplified; optionally converts to half precision, halving download size; serves the model as a static same-origin asset; and executes it in a dedicated worker thread through ONNX Runtime Web, using the WebGPU execution provider where available and multi-threaded WebAssembly as a fallback.

Cross-origin isolation headers are required on every response, because multi-threaded WebAssembly depends on SharedArrayBuffer, which is unavailable to a page that is not cross-origin isolated.

Two models were selected for deployment - YOLO26n at 480 pixels and YOLO26n at 960 pixels - forming a two-tier system in which the lighter model is the default and the heavier one an explicit higher-quality option. The measurements supporting that selection are presented in Chapter (4).

Chapter (4)

Implementation and Results

4.1 Experimental Setup and Environment

This chapter presents the experimental programme and its results: the environment in which the work was carried out, the training runs performed, the comparative accuracy of the architectures evaluated, the measured performance characteristics of the candidate models, and the deployed system.

All training, evaluation and benchmarking was performed on a single workstation with an NVIDIA RTX 2000 Ada Generation GPU providing 17.18 GB of VRAM, running Windows. The software environment was Python 3.11.15, PyTorch 2.6.0 with CUDA 12.4, and Ultralytics 8.4.64. Browser deployment uses ONNX Runtime Web 1.27.0.

One platform constraint shaped the training configuration. Data-loader worker counts above two produced repeated process failures on this system, and two workers was the only value verified stable across every batch size used. This limited data-loading throughput and therefore influenced achievable batch sizes, but did not affect model quality.

4.2 Training Programme and Computational Expenditure

Twenty-one training runs were completed across six model configurations in four architecture families, totalling 950 epochs and 277.57 GPU-hours. Runs varied in architecture, capacity, input resolution, augmentation intensity, and initialisation strategy.

Two properties of the training record require explicit statement. First, wall-clock duration for ten of the runs is self-reported from run summaries rather than machine-logged; these are marked in the results table, and the two categories are not aggregated into a single unqualified total. Second, per-epoch training histories were preserved for fifteen of the twenty-one runs. For the remaining six - the five RF-DETR runs and one YOLO11m run - no training log survives in any form, and the checkpoints themselves store only a final epoch counter rather than a loss series. Loss curves are therefore presented for fifteen runs and the absence is recorded for the other six, rather than filled with a substitute.

Table (4.1) Training effort by architecture family

Family	Runs	Epochs	GPU-hours	Best mAP@0.5 (%)
YOLO11m	4	148	81.98	86.75
YOLO11n	3	232	37.19	83.11
YOLO26n	8	386	65.61	82.75
YOLO26s	1	47	20.42	81.60
RF-DETR-Nano	2	65	16.34	78.99
RF-DETR-Small	3	72	56.03	73.55
Total	21	950	277.57	-

4.3 Comparative Accuracy

All models were evaluated on the same held-out partition of 5,589 images containing 7,427 instances, at IoU 0.5 and operating confidence 0.35. Both raw and label-gap-corrected mAP@0.5 are reported; the correction, described in Section 3.6.2, ignores predictions of a class that the contributing source family never annotates, and by construction leaves recall unchanged.

Table (4.2) All training runs, ranked by corrected mAP@0.5

#	Model	Family	Input	mAP raw (%)	mAP corr. (%)	P (%)	R (%)	F1 (%)	Epochs	GPU-h	Timing
1	YOLO11m Warm-Start	YOLO11m	640	86.42	86.75	82.69	73.82	77.97	15	7.23	logged
2	YOLO11n Capacity	YOLO11n	960	82.73	83.11	82.23	71.93	76.61	112	23.07	logged
3	YOLO26n Calibration	YOLO26n	960	82.34	82.75	78.99	73.37	76.06	40	9.50	logged
4	YOLO26n Fine-Tune	YOLO26n	960	82.33	82.73	79.64	73.48	76.41	50	11.70	logged
5	YOLO26n Weak-Device	YOLO26n	480	82.28	82.72	79.30	72.31	75.61	25	1.63	logged
6	YOLO26n Class-Weight 3.0	YOLO26n	960	81.79	82.20	78.92	72.56	75.59	34	8.08	logged
7	YOLO26s Capacity	YOLO26s	960	81.17	81.60	81.85	64.07	71.12	47	20.42	logged
8	YOLO26n Fresh 640	YOLO26n	640	81.02	81.46	78.95	69.84	74.01	100	10.73	logged

9	YOLO26n Baseline	YOLO26n	960	79.55	79.76	75.33	72.34	73.79	77	20.39	logged
10	RF-DETR-Nano Fine-Tune	RF-DETR-Nano	384	78.29	78.99	62.34	85.90	72.10	15	11.84	reported
11	RF-DETR-Nano Baseline	RF-DETR-Nano	384	78.18	78.98	61.37	86.47	71.62	50	4.50	reported
12	YOLO11m Cross-Dataset	YOLO11m	640	75.63	76.35	75.86	70.54	72.95	40	40.43	reported
13	YOLO11m Extended	YOLO11m	640	73.94	74.60	76.01	67.89	71.53	53	26.64	reported
14	RF-DETR-Small Worst-Case	RF-DETR-Small	384	72.66	73.55	60.76	81.39	69.48	17	8.50	approx.
15	RF-DETR-Small Standard	RF-DETR-Small	640	72.09	73.12	60.77	79.57	68.77	40	33.33	reported
16	YOLO11m Worst-Case	YOLO11m	384	72.30	72.89	72.35	71.19	71.67	40	7.68	reported
17	YOLO11n Worst-Case	YOLO11n	640	71.98	72.77	74.51	67.10	70.56	60	7.01	reported
18	YOLO11n Baseline	YOLO11n	384	69.64	70.32	76.71	62.93	68.83	60	7.11	reported
19	YOLO26n Compact Worst-Case	YOLO26n	384	69.21	69.97	70.59	67.19	68.77	20	1.06	logged
20	YOLO26n Compact Baseline	YOLO26n	384	68.13	68.80	72.89	63.05	67.46	40	2.52	logged
21	RF-DETR-Small Baseline	RF-DETR-Small	640	65.29	66.21	60.21	73.49	66.02	15	14.20	reported

Timing column: 'logged' indicates machine-recorded duration; 'reported' indicates a figure self-reported in a run summary; 'approx.' indicates a prose approximation. These are not aggregated as a single unqualified total.

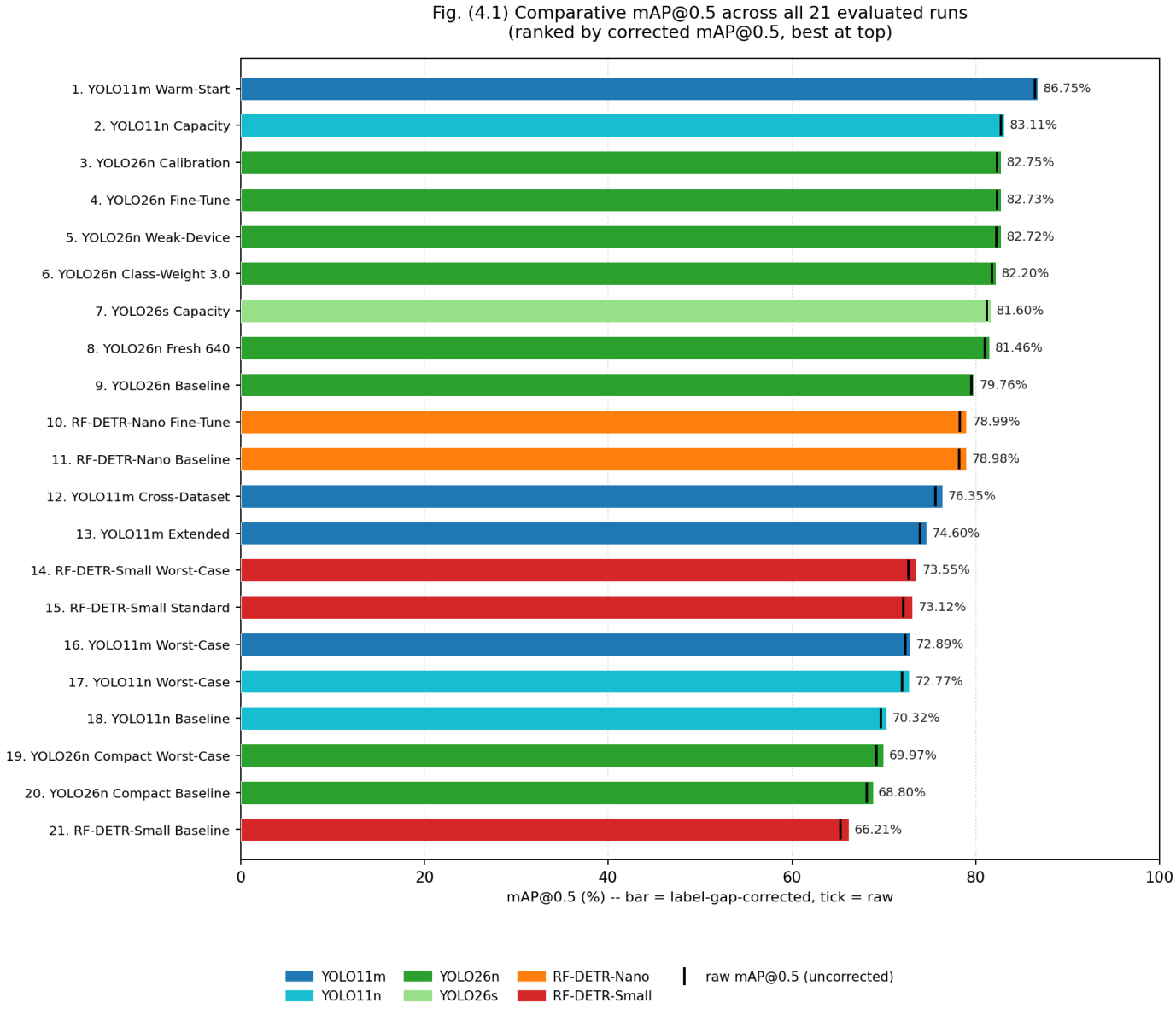

Fig. (4.1) Comparative mAP@0.5 across all evaluated runs

Fig. (4.2) Per-class average precision across evaluated runs

Three results merit specific comment.

The highest accuracy in the study was obtained by a warm-started YOLO11m run in 15 epochs and 7.23 GPU-hours. Several runs trained for far longer reached lower accuracy - one YOLO11m configuration consumed 40.43 GPU-hours across 40 epochs to reach a markedly lower score. Initialisation strategy, not training duration, was the dominant factor.

The transformer-based RF-DETR models did not outperform the single-stage convolutional models on this task, despite substantially greater capacity: RF-DETR-Nano carries 30.15 M parameters against YOLO26n's 2.50 M, an order of magnitude more, for lower accuracy. Architectural novelty did not translate into advantage here.

The three highest-scoring YOLO26n configurations are separated by 0.03 percentage points of corrected mAP@0.5. That difference is not meaningful, and no ranking claim is made between them on the basis of it. Their selection for deployment rests on the size and latency measurements presented below, not on their ordering in this table.

4.4 The Deployed Configurations

Table (4.3) The two deployed configurations compared

Property	YOLO26n 480	YOLO26n 960
Corrected mAP@0.5 (%)	82.72	82.75
Precision (%)	79.30	78.99
Recall (%)	72.31	73.37
F1 (%)	75.61	76.06
AP closed_eye (%)	86.39	88.69
AP open_eye (%)	83.00	83.27
AP yawning (%)	78.77	76.28

Aggregate mAP conceals a trade between the two deployed configurations that is directly relevant to the application. The 960-pixel model is stronger on closed_eye, the class from which microsleep evidence is derived; the 480-pixel model is stronger on yawning. Because these classes carry different safety weight - a missed microsleep is a more serious failure than a missed yawn - the two models were deployed as an explicit two-tier choice rather than treating either as a replacement for the other.

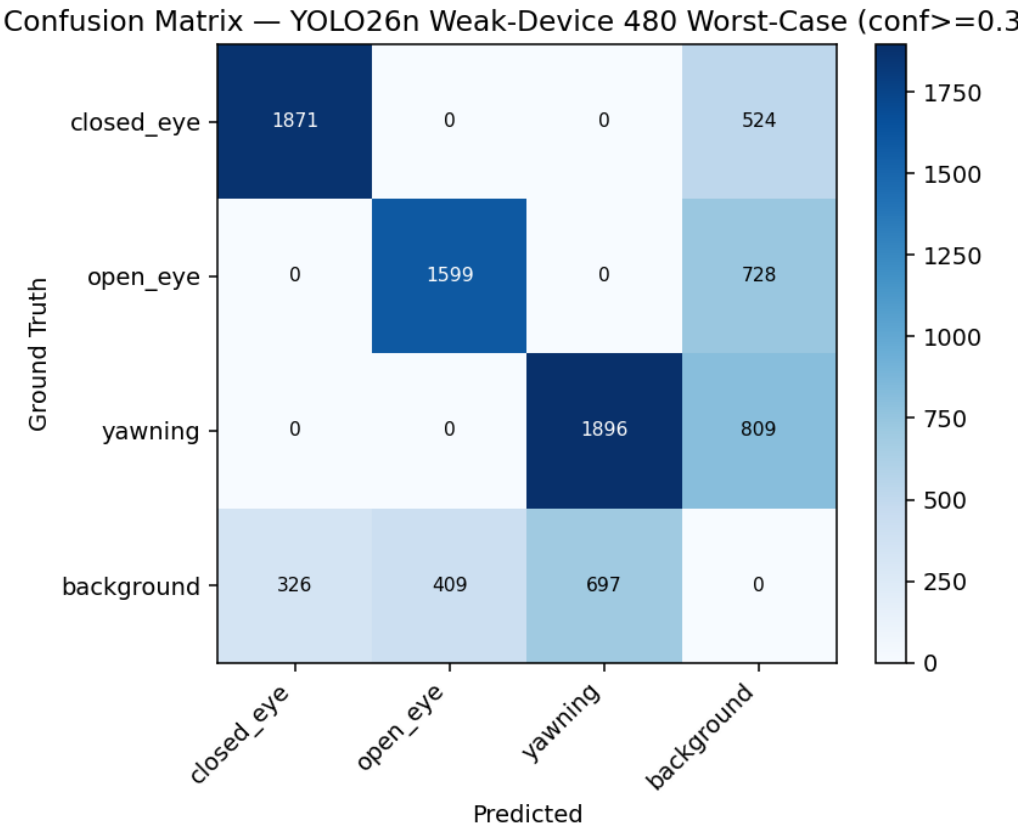

Fig. (4.3) Confusion matrix, deployed 480-pixel configuration

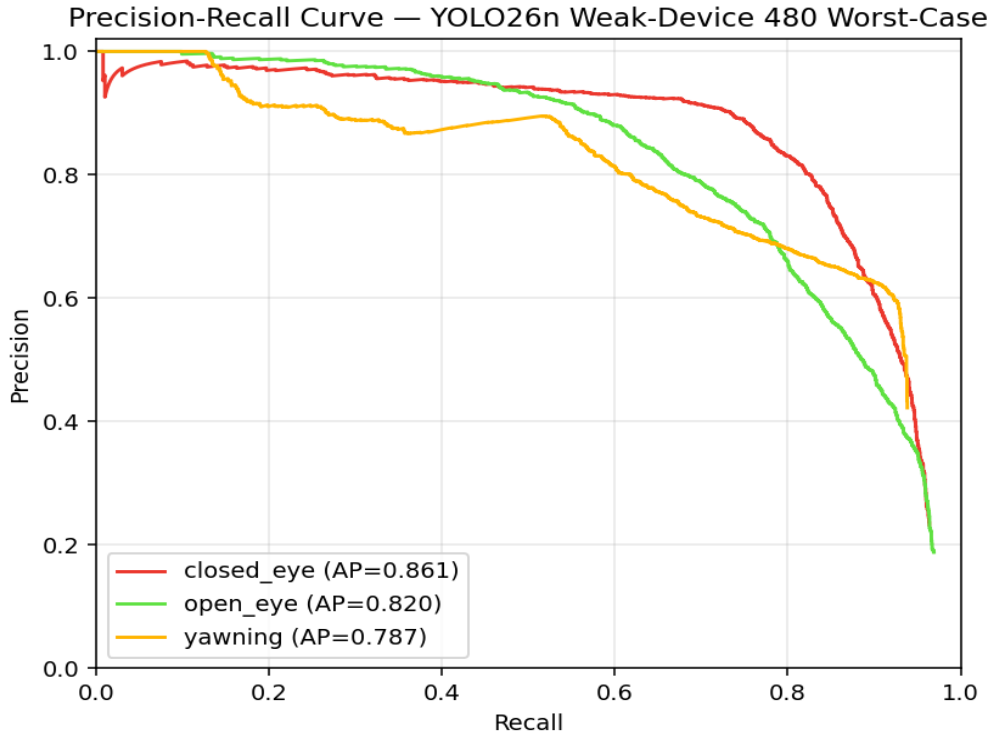

Fig. (4.4) Precision-recall curves, deployed 480-pixel configuration

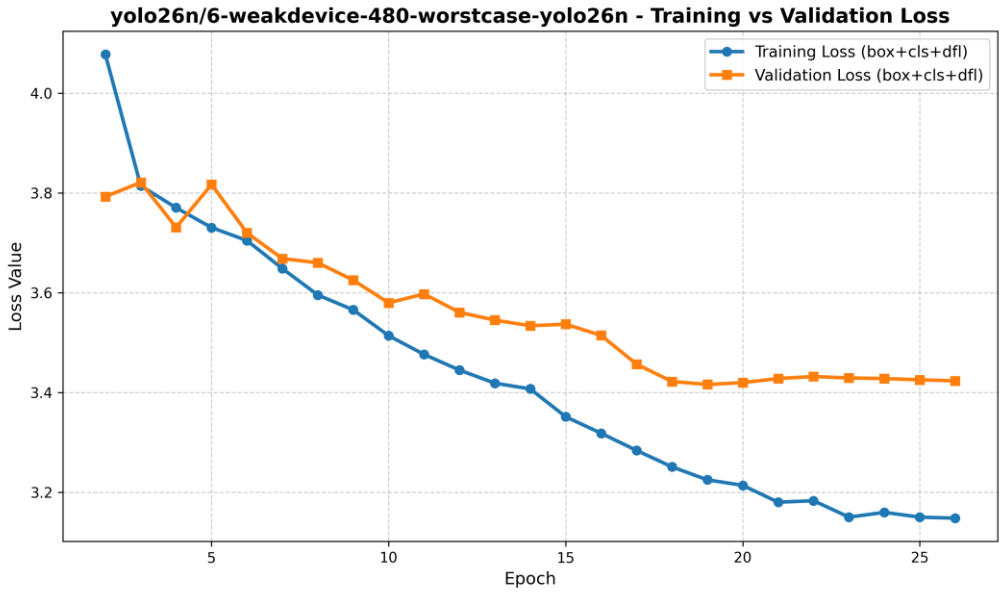

Fig. (4.5) Training and validation loss, deployed 480-pixel configuration

4.5 Measured Inference Performance

Inference latency was measured directly rather than inferred from input dimensions. Each configuration was timed over 50 iterations at batch size 1 after 10 discarded warm-up iterations, with CUDA synchronisation on both sides of every timed region. Synchronisation is essential: CUDA kernel launches are asynchronous, and timing them without it measures the speed of instruction dispatch rather than of computation. Reported throughput is derived from the median rather than the mean, so that a single outlier cannot distort it. The measurement covers the forward pass on a synthetic input of each model's own size and excludes pre-processing and post-processing, and is therefore a lower bound on end-to-end cost.

Table (4.4) Measured inference latency and model size

Model	Input	Params (M)	Size (MB)	GPU median (ms)	GPU FPS	CPU median (ms)	CPU FPS
YOLO11m Warm-Start	640	20.05	38.6	11.26	88.81	151.67	6.59
YOLO11n Capacity	960	2.59	20.4	6.98	143.23	69.44	14.40
YOLO26n Calibration	960	2.50	5.2	21.57	46.35	401.28	2.49
YOLO26n Fine-Tune	960	2.50	5.2	10.69	93.55	83.67	11.95
YOLO26n Weak-Device	480	2.50	5.1	10.29	97.20	49.75	20.10
YOLO26n Class-Weight 3.0	960	2.50	5.2	10.18	98.20	88.01	11.36

YOLO26s Capacity	960	9.95	76.7	12.27	81.51	162.53	6.15
YOLO26n Fresh 640	640	2.50	5.1	21.26	47.04	169.19	5.91
YOLO26n Baseline	960	2.50	14.9	11.03	90.63	84.86	11.78
RF-DETR-Nano Fine-Tune	384	30.15	115.2	11.29	88.55	81.30	12.30
RF-DETR-Nano Baseline	384	30.15	115.2	13.90	71.96	81.33	12.30
YOLO11m Cross-Dataset	640	20.05	38.6	10.54	94.89	154.79	6.46
YOLO11m Extended	640	20.05	115.2	10.50	95.21	154.85	6.46
RF-DETR-Small Worst-Case	384	31.63	120.8	14.18	70.53	87.63	11.41
RF-DETR-Small Standard	640	32.02	122.3	18.70	53.49	170.53	5.86
YOLO11m Worst-Case	384	20.05	38.6	9.43	106.01	81.18	12.32
YOLO11n Worst-Case	640	2.59	5.2	7.18	139.29	46.59	21.47
YOLO11n Baseline	384	2.59	5.2	7.43	134.64	30.80	32.47
YOLO26n Compact Worst-Case	384	2.50	5.1	10.95	91.31	40.52	24.68
YOLO26n Compact Baseline	384	2.50	5.1	11.19	89.39	38.58	25.92
RF-DETR-Small Baseline	640	32.02	122.3	18.73	53.38	170.97	5.85

The measurements contradicted an assumption carried through the earlier part of this work. Reducing input resolution from 960 to 480 pixels reduces the number of input pixels by a factor of four, and the computational cost of convolution scales with that count. It was expected that latency would fall correspondingly. It did not. Across the YOLO26n configurations, GPU latency is approximately constant at 384, 480, 640 and 960 pixel inputs.

The explanation is that at batch size 1 with a 2.50 M-parameter model, GPU execution is dominated by fixed per-launch overhead rather than by arithmetic. The GPU is not saturated, so reducing the work does not reduce the elapsed time. The advantage of the smaller input appears on CPU, where computation genuinely dominates and the relationship holds.

This has a direct consequence for deployment reasoning: on capable GPU hardware, the smaller model's benefit is download size rather than speed, while on CPU-bound devices the latency benefit

is real. It also illustrates why the measurement was necessary - the expectation derived from first principles was reasonable and was wrong.

A second measured result concerns model size rather than speed. RF-DETR-Nano requires 115 MB on disk against YOLO26n's 5.11 MB, a factor of twenty-two, while being only marginally slower on GPU. For a model that must be downloaded to a browser before the first inference can run, size rather than latency is the binding constraint, and it is on that basis that the transformer models were excluded from deployment.

4.6 Export and Precision Conversion

The two selected configurations were exported to ONNX at their own training resolutions, with non-maximum suppression compiled into the graph and the graph structurally simplified. The exported models emit a fixed tensor of 300 candidate detections, each carrying box coordinates, a confidence score and a class index, already suppressed. This removes an entire post-processing stage from the browser client.

Both were additionally converted to half precision to reduce download size. Because reduced numerical precision can in principle alter detection behaviour, the converted models were verified against their full-precision counterparts on real test images rather than assumed equivalent. Detections were matched between precisions by spatial overlap and compared. The 960-pixel model produced identical detection counts with no class disagreements and a maximum box displacement of 0.64 pixels; the 480-pixel model matched on all common detections with a maximum displacement of 0.36 pixels, differing by one additional detection near the confidence threshold. Half-precision conversion is therefore behaviourally lossless at this scale while halving model size.

4.7 Deployed System

The deployed system executes the exported models entirely within the user's browser. Inference runs in a dedicated worker thread so that the user interface is not blocked during processing, using the WebGPU execution provider where the device supports it and falling back to multi-threaded WebAssembly otherwise. Multi-threaded WebAssembly requires SharedArrayBuffer, which is available only to a cross-origin-isolated page, so the application sets the required isolation headers on every response; without them the runtime silently degrades to single-threaded execution.

Per-frame detections are aggregated by the temporal layer described in Section 3.3 into a rolling PERCLOS estimate and a debounced driver state. No video frame is transmitted from the device at any point.

The two models are presented to the user as a two-tier choice. The 480-pixel configuration is the default, on the basis that it matches the larger model's aggregate accuracy while being cheaper on

CPU-bound hardware; the 960-pixel configuration is offered as a higher-quality option where its stronger closed_eye performance is preferred and the hardware permits it.

These are drawn from the deployed 480px model's own Ultralytics validation-batch prediction grids, generated during training - genuine model output on real corpus images, not staged or cherry-picked beyond excluding a handful of synthetic-iris augmentation artefacts. They are validation-split images rather than the held-out test split used for the metrics reported elsewhere in this chapter; the held-out-test evaluation script itself only renders a fixed 4-image sample, which is too few to illustrate the range of conditions the deployed model was trained on.

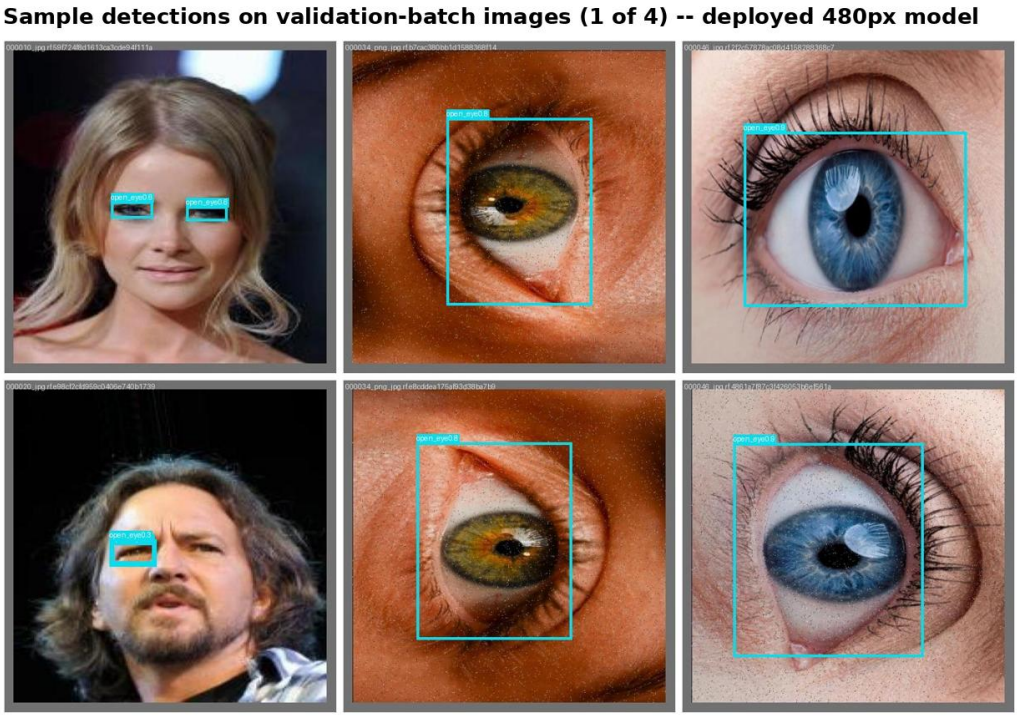

Fig. (4.6) Sample detections, deployed 480px model, validation-batch images (1 of 4)

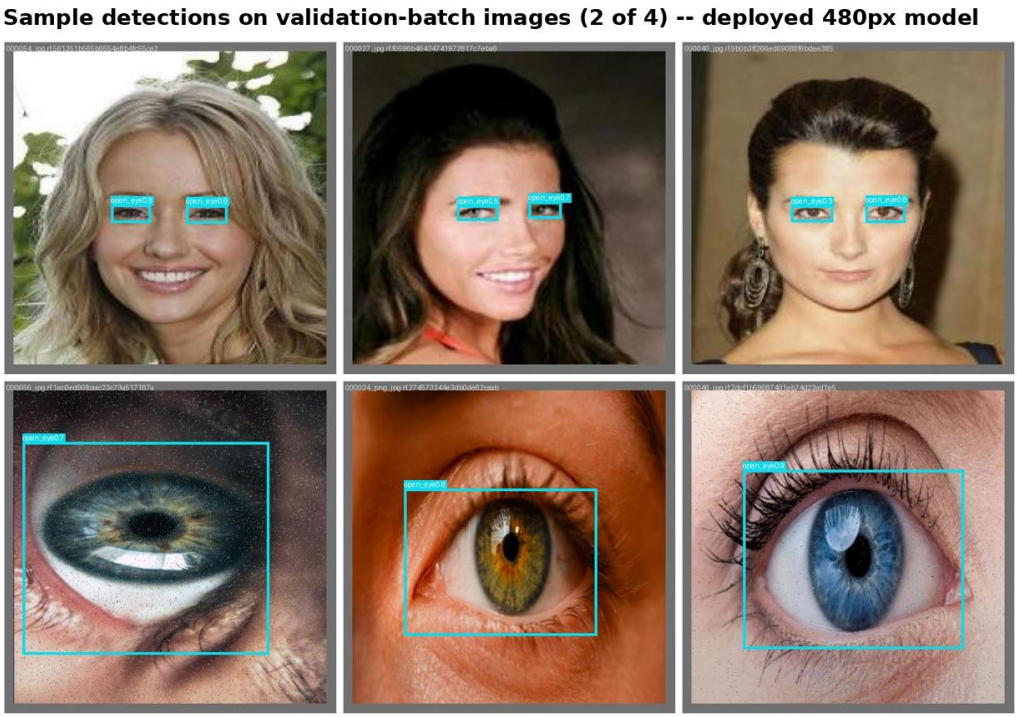

Fig. (4.7) Sample detections, deployed 480px model, validation-batch images (2 of 4)

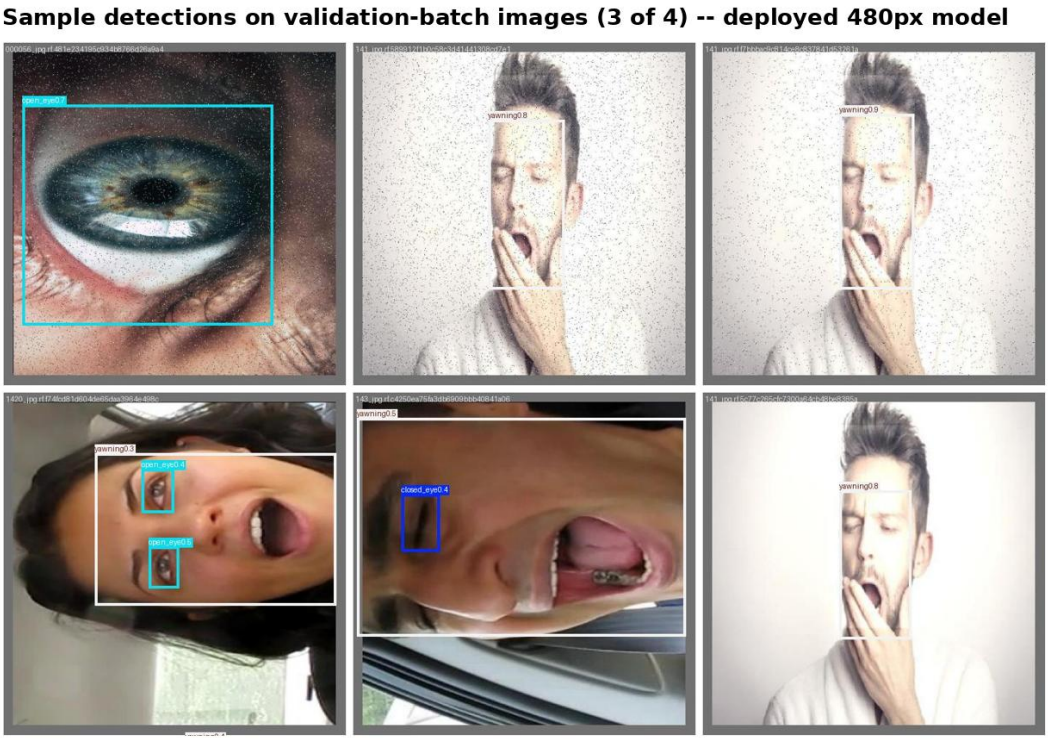

Fig. (4.8) Sample detections, deployed 480px model, validation-batch images (3 of 4)

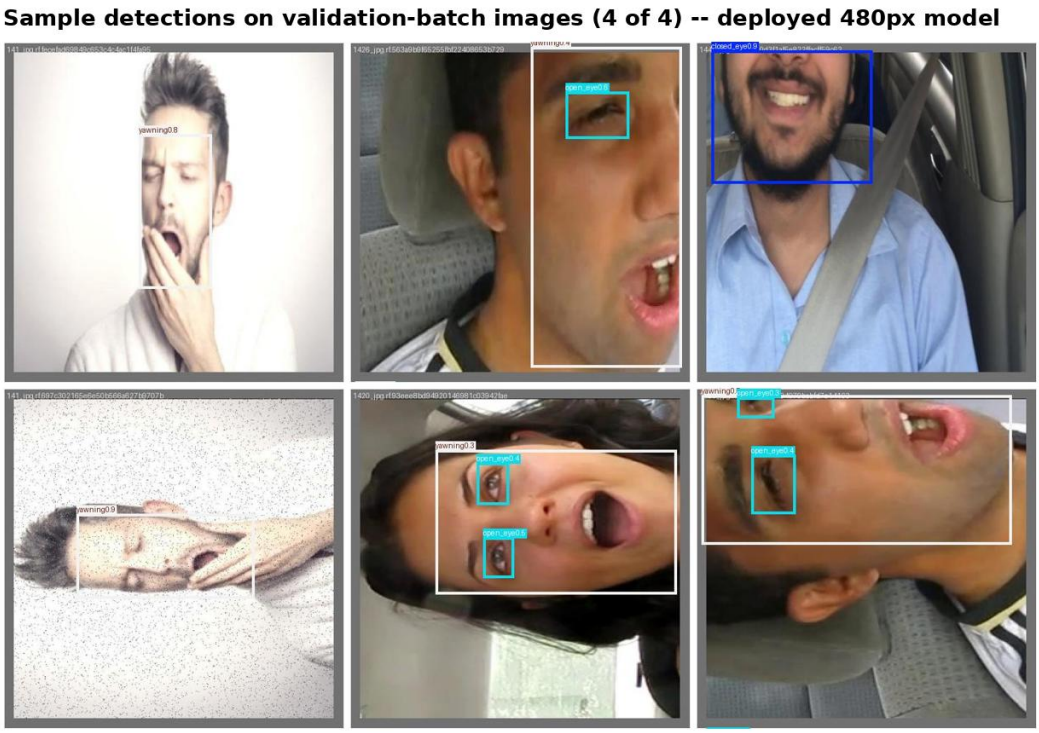

Fig. (4.9) Sample detections, deployed 480px model, validation-batch images (4 of 4)

4.8 From-Scratch Faster R-CNN: Separate Evaluation

A simplified Faster R-CNN was implemented from first principles, without a detection framework, as described in Section 3.4.4. Its results are presented separately here because it was evaluated on a test partition of 5,705 images rather than the 5,589-image partition used for every other model in this study. The two sets of figures are therefore not directly comparable, and no ranking between them is implied.

Three configurations were evaluated: a baseline, a tuned configuration, and a further refined configuration. Tuning improved mAP@0.5 from 72.61 % to 74.27 % and mAP@0.5:0.95 from 32.76 % to 34.60 %. The per-class pattern is informative: yawning improved by 3.62 percentage points and open_eye by 2.87, while closed_eye declined by 1.49. An automated pass over the training split then flagged 668 high-confidence detections (≥95 % confidence) that did not overlap any existing ground-truth box — objects the model itself found but the original labels had missed. After human review, 611 were approved as genuine missing labels and appended to the training set, and 15 were rejected as false detections. Training was then resumed from the tuned checkpoint for 15 further epochs on the completed labels (to epoch 65, best at epoch 63), producing the refined configuration. mAP@0.5 rose further to 77.84 % and mAP@0.5:0.95 to 36.90 %, driven mainly by recall, which improved from 82.60 % to 86.02 % as previously unlabelled objects were now correctly supervised.

The value of this implementation is primarily educational. Constructing a region proposal network, anchor generation, RoI alignment and a two-stage training loop with four simultaneous loss

terms from scratch establishes an understanding of detection mechanics that using a framework does not. Its accuracy is below that of the framework-based single-stage models evaluated in this study, which is the expected outcome and not the purpose of the exercise.

Table (4.5) From-scratch Faster R-CNN results (5,705-image partition; not comparable with Table 4.2)

Metric	Baseline	Tuned	Refined
mAP@0.5 (%)	72.61	74.27	77.84
mAP@0.5:0.95 (%)	32.76	34.60	36.90
Precision (%)	70.62	71.02	71.30
Recall (%)	82.47	82.60	86.02
F1 (%)	76.09	76.37	77.97
AP closed_eye (%)	74.65	73.15	77.48
AP open_eye (%)	64.70	67.57	72.09
AP yawn (%)	78.47	82.10	83.95

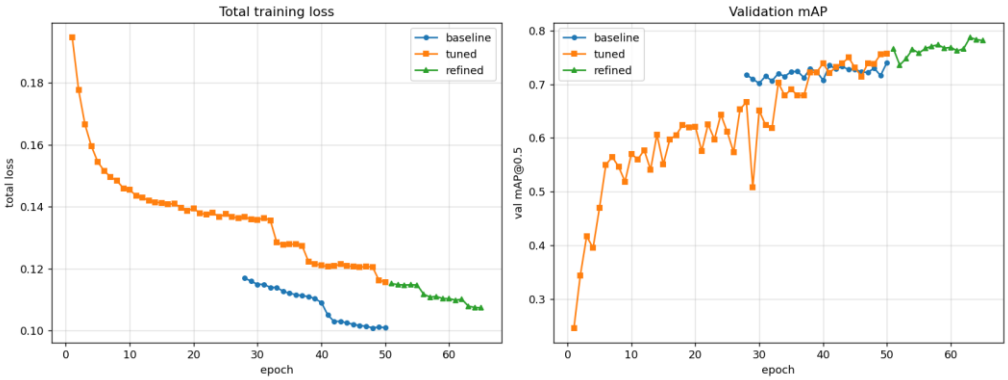

Fig. (4.10) From-scratch Faster R-CNN: baseline, tuned, and refined configurations

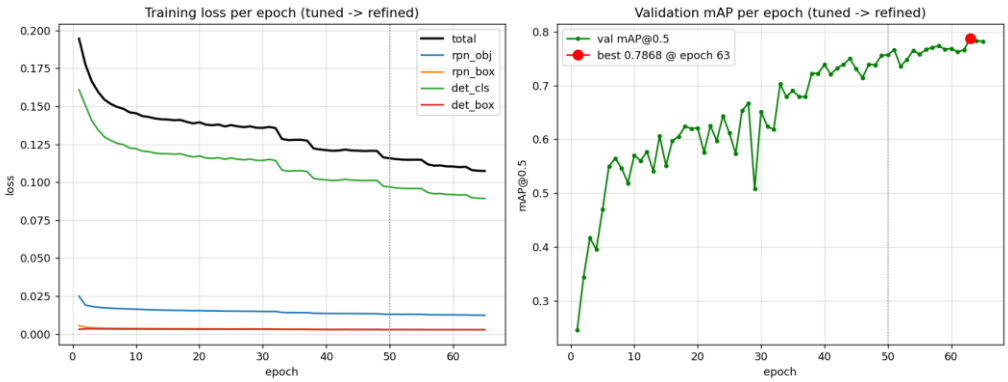

Fig. (4.11) From-scratch Faster R-CNN: four-component training loss

4.9 Confusion Matrices and Loss Curves — Best Model per Architecture Family

This section gathers the most diagnostically important figures for the best-performing configuration in each of the six architecture families trained during this project (YOLO11m, YOLO11n, YOLO26n, YOLO26s, RF-DETR-Nano, RF-DETR-Small), so the four-way family comparison in Section 4.5 and Table (4.2) can be read alongside each family's own confusion matrix and training curve. All six are the real, unedited output of this project's own evaluation script (confusion matrices) and training logs (loss curves) for the exact checkpoint ranked in Table (4.2); none of the values shown are estimated.

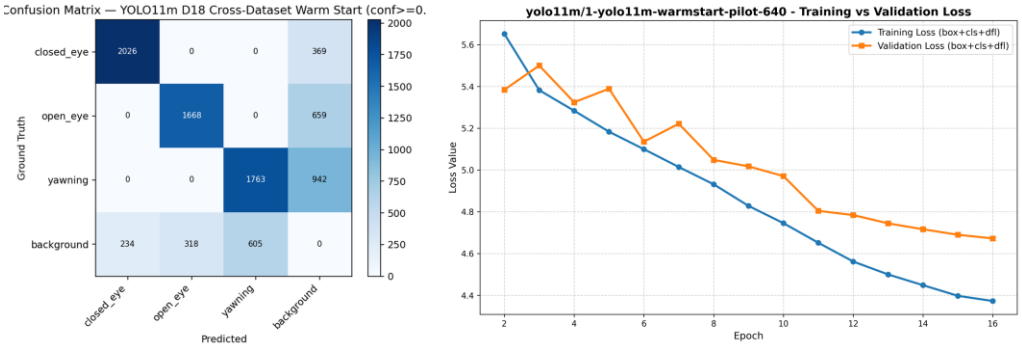

Fig. (4.12) YOLO11m Warm-Start (rank 1 overall): confusion matrix and training/validation loss

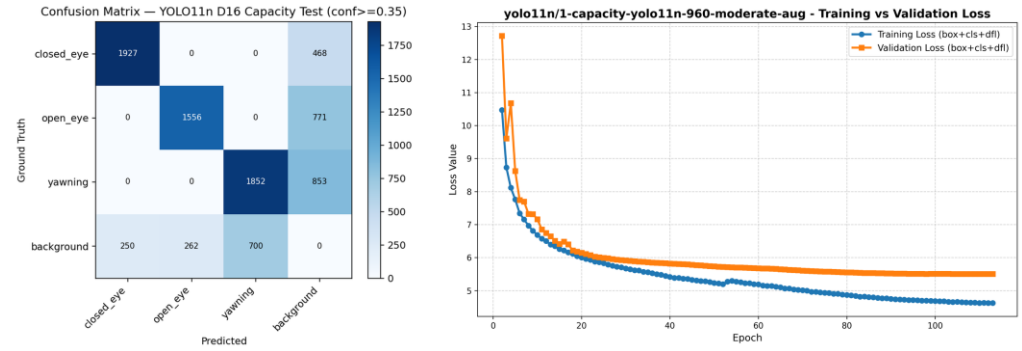

Fig. (4.13) YOLO11n Capacity (best YOLO11n run, rank 2 overall): confusion matrix and training/validation loss

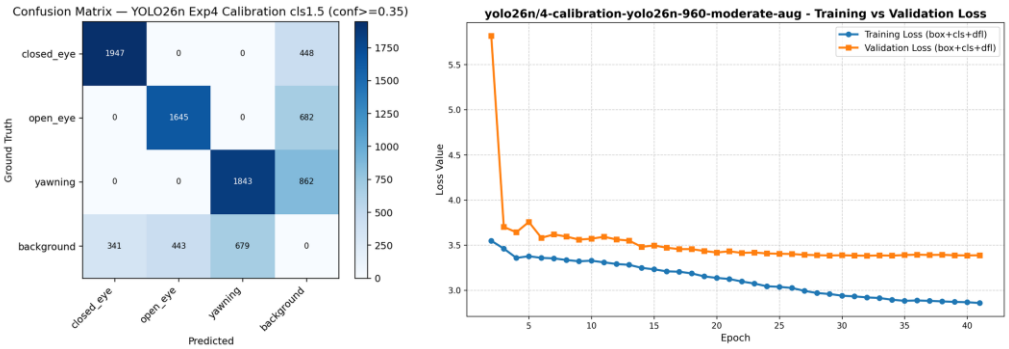

Fig. (4.14) YOLO26n Calibration/960px (best YOLO26n run, rank 3 overall, deployed high tier): confusion matrix and training/validation loss

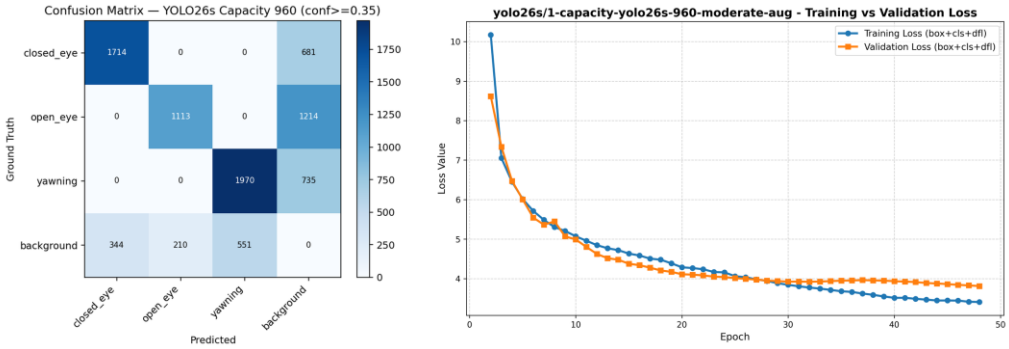

Fig. (4.15) YOLO26s Capacity (only YOLO26s run, rank 7 overall): confusion matrix and training/validation loss

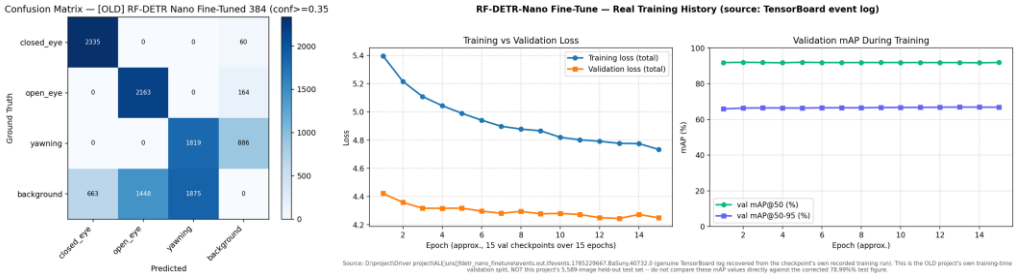

Fig. (4.16) RF-DETR-Nano Fine-Tune (best RF-DETR-Nano run, rank 10 overall): confusion matrix and recovered training/validation loss

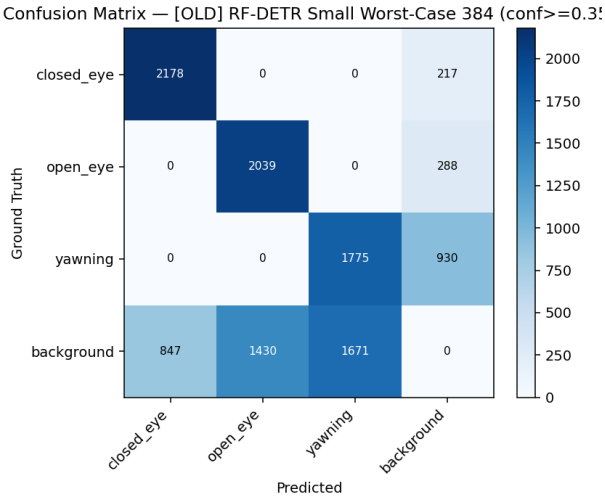

Fig. (4.17) RF-DETR-Small Worst-Case (best RF-DETR-Small run, rank 14 overall): confusion matrix

No per-epoch training log survives for this run, unlike the other five in this set, so only its confusion matrix is shown.

4.10 Summary Comparison Across All Evaluated Runs

Table (4.6) compares the five configurations directly. The from-scratch Faster R-CNN row is marked as evaluated on a different test set and should not be read as directly ranked against the other four.

Table (4.6) Comparison of the five detailed model records

	YOLO26n 480	YOLO26n 960	RF-DETR Nano-FT	YOLO11m Warm-Start	F-RCNN (scratch)
Test set	5,589 img	5,589 img	5,589 img	5,589 img	5,705 img*
mAP@0.5 (corr.)	82.72%	82.75%	78.99%	86.75%	74.27%
mAP@0.5:0.95	N/A	N/A	N/A	N/A	34.60%
Precision	79.30%	78.99%	62.34%	82.69%	71.02%
Recall	72.31%	73.37%	85.90%	73.82%	82.60%
F1	75.61%	76.06%	72.10%	77.97%	76.37%
AP closed_eye	86.39%	88.69%	97.69%	92.62%	73.15%
AP open_eye	83.00%	83.27%	82.02%	87.01%	67.57%
AP yawning/yawn	78.77%	76.28%	57.27%	80.61%	82.10%
Params	2.50M	2.50M	30.15M	20.06M	16.86M
Checkpoint size	5.11 MB	5.20 MB	115.2 MB	38.64 MB	128.6 MB
Deployed ONNX	9.28/4.71 MB	9.55/4.84 MB	108.1 MB	not deployed	not deployed
GPU latency	10.29 ms	unreliable*	11.29 ms	11.26 ms	N/A
CPU latency	49.75 ms	unreliable*	81.30 ms	151.67 ms	N/A
Epochs	25	40	15	15	50
Deployed in production	yes (default)	yes (opt-in)	no	no	no

* The from-scratch Faster R-CNN's test set differs in size from the other four and its result is not directly comparable. The YOLO26n 960px GPU/CPU latency figures were flagged unreliable in Section A.3.2 above and are marked accordingly rather than quoted as measured.

The five records above are drawn from a wider sweep of 21 trained runs across four architecture families, already ranked in full in Table (4.2). Table (4.7) reproduces the seventeen runs from that sweep not already detailed individually above, to keep this appendix's own record of the search effort self-contained.

Table (4.7) The remaining seventeen runs of the 21-run architecture and hyperparameter sweep

#	Model	Family	Input	mAP corr.(%)	P(%)	R(%)	F1(%)	Epochs
2	YOLO11n Capacity	YOLO11n	960	83.11	82.23	71.93	76.61	112
4	YOLO26n Fine-Tune	YOLO26n	960	82.73	79.64	73.48	76.41	50
6	YOLO26n Class-Weight 3.0	YOLO26n	960	82.20	78.92	72.56	75.59	34
7	YOLO26s Capacity	YOLO26s	960	81.60	81.85	64.07	71.12	47
8	YOLO26n Fresh 640	YOLO26n	640	81.46	78.95	69.84	74.01	100
9	YOLO26n Baseline	YOLO26n	960	79.76	75.33	72.34	73.79	77
11	RF-DETR-Nano Baseline	RF-DETR-Nano	384	78.98	61.37	86.47	71.62	50
12	YOLO11m Cross-Dataset	YOLO11m	640	76.35	75.86	70.54	72.95	40
13	YOLO11m Extended	YOLO11m	640	74.60	76.01	67.89	71.53	53
14	RF-DETR-Small Worst-Case	RF-DETR-Small	384	73.55	60.76	81.39	69.48	17
15	RF-DETR-Small Standard	RF-DETR-Small	640	73.12	60.77	79.57	68.77	40
16	YOLO11m Worst-Case	YOLO11m	384	72.89	72.35	71.19	71.67	40
17	YOLO11n Worst-Case	YOLO11n	640	72.77	74.51	67.10	70.56	60
18	YOLO11n Baseline	YOLO11n	384	70.32	76.71	62.93	68.83	60
19	YOLO26n Compact Worst-Case	YOLO26n	384	69.97	70.59	67.19	68.77	20
20	YOLO26n Compact Baseline	YOLO26n	384	68.80	72.89	63.05	67.46	40
21	RF-DETR-Small Baseline	RF-DETR-Small	640	66.21	60.21	73.49	66.02	15

Ranks and figures are reused unchanged from Table (4.2); rank 1, 3, 5 and 10 of that table are the YOLO11m warm-start, YOLO26n calibration (960px), YOLO26n weak-device (480px) and RF-DETR-Nano fine-tune runs detailed individually in Sections A.3.1-A.3.4 above and are not repeated in this table. Twenty of the twenty-one runs are logged from Ultralytics-native results.csv files or reported test-time metrics; one (RF-DETR-Small Worst-Case) is marked approximate in its source record.

Chapter (5)

Conclusion

5.1 Conclusions

This project set out to build a camera-only driver drowsiness detection system capable of running on a user's own device, and to do so on evidence that could be independently checked. Both the system and the evidence base were delivered.

On dataset construction, the central finding was that the working corpus was not one dataset but several single-task datasets merged into a shared label space without re-annotation. This produced missing supervision that is systematic and correlated with the source of each image, not random noise. Recognising this changed both training and evaluation: compositing augmentations were structurally excluded because they destroy source identity, and a corrected evaluation metric was introduced that ignores predictions of classes a source family never annotated. The final dataset comprises 50,654 images and 68,292 instances, partitioned group-aware and independently verified against leakage.

On architecture selection, twenty-one runs across four families showed that architectural novelty did not automatically produce advantage on this task. The transformer-based RF-DETR models, despite being an order of magnitude larger in parameter count than the YOLO26n models, did not outperform them. The strongest single accuracy result came from a warm-started YOLO11m run that reached the highest mAP@0.5 in the study in only 15 epochs, demonstrating that initialisation strategy mattered more than training duration.

On the resolution question, the finding was more consequential than expected. Reducing input resolution from 960 to 480 pixels cost essentially nothing in aggregate accuracy - 82.75 % against 82.72 % corrected mAP@0.5 - but the assumption that this would translate into a proportional latency saving proved false when measured. On GPU at batch size one, inference is dominated by fixed overhead rather than by pixel count, and the two resolutions perform comparably. The measurable advantage of the

smaller input appears on CPU, where computation genuinely dominates. This is a case where a plausible inference from first principles was contradicted by direct measurement, and it is recorded as such.

The aggregate figures also concealed a per-class trade that matters for the application: the 960-pixel model is meaningfully stronger on closed_eye, the class from which microsleep evidence is derived, while the 480-pixel model is stronger on yawning. Both models were therefore deployed as an explicit two-tier choice rather than one being declared a replacement for the other.

On deployment, half-precision conversion halved model size with no meaningful behavioural change, verified by matching detections between precisions on real test images rather than assumed from the numerical properties of the format.

Finally, the project treated evidence integrity as a technical requirement. Inherited evaluation records from an earlier phase were found to contain chart images duplicated across models that could not possibly share them, and all results were consequently re-measured under a single protocol and verified distinct. Where an artefact could not be honestly reconstructed, its absence was recorded rather than substituted.

5.2 Limitations

The dataset is assembled from heterogeneous sources rather than collected under controlled conditions, and its images are predominantly still photography rather than continuous in-vehicle video. Performance on sustained real driving footage is therefore not established by the results presented here.

Training histories for six of the twenty-one runs were not preserved and could not be reconstructed, so loss curves are absent for those runs. The models themselves were re-evaluated directly from their checkpoints and are unaffected.

The from-scratch Faster R-CNN was evaluated on a test partition of 5,705 images rather than the 5,589-image partition used for all other models. Its results are reported separately and are not directly comparable.

Latency was measured on desktop hardware using PyTorch and ONNX Runtime. Browser WebGPU performance, which is what the deployed system actually experiences, was not measured within the scope of this document.

Training duration figures for ten of the runs are self-reported from run summaries rather than machine-logged, and are labelled as such wherever they appear.

5.3 Future Work

The most direct extension is learned temporal modelling. The present system aggregates per-frame detections using a rolling-window PERCLOS estimate with debounced transitions, which is effective but hand-designed. Recurrent or sequence architectures trained directly on temporal sequences would allow the fatigue signal itself to be learned rather than specified.

Second, in-browser latency measurement on real target devices, particularly low-end mobile hardware, would replace the desktop proxy figures reported here with measurements from the deployment environment.

Third, integer quantisation was deliberately deferred in this work pending finalisation of the model selection. With deployment candidates now fixed, calibrated INT8 quantisation is a realistic next step for further reducing model size, subject to per-class accuracy verification.

Fourth, evaluation against an independently collected in-vehicle video dataset would test generalisation beyond the merged corpus used here.

Appendix (A)

A.1 Complete Experimental Record

The complete per-model evidence base - evaluation reports, metrics, ten diagnostic charts per model, and training curves where preserved - is retained in the project's INFO directory, organised by architecture family and run. Table (4.2) summarises all twenty-one runs.

A.2 Reproducibility

Training configurations for every run are retained as version-controlled YAML files. The evaluation, benchmarking and chart-generation procedures are implemented as scripts in the project's src directory and were used to produce every figure and table in this document.

A.3 Detailed Model Records

This section documents the five architecturally distinct configurations that anchor the comparison in Chapter 4 - the two deployed YOLO26n variants, the best-performing configuration from each of the two non-deployed architecture families, and the tuned from-scratch Faster R-CNN - with their checkpoint and configuration file locations, full training and evaluation protocol, and evidence figure, so that every reported number in this document can be traced back to a specific run. A comparison table follows the five records, and Table (A.2) then summarises the remaining seventeen runs of the twenty-one-run sweep, reusing the figures already reported in Table (4.2), to show the extent of the search that preceded the final architecture and hyperparameter choices.

A.3.1 YOLO26n — 480px (Deployed, Low Tier)

This is the low-tier deployed configuration reported in Section 4.4-4.6 (Fig. 4.3-4.5). The checkpoint and its training configuration are retained at checkpoints/yolo26n/6-weakdevice-480-worstcase-yolo26n/best.pt and .../args.yaml. Evaluated on the project's own 5,589-image held-out test set at IoU 0.5 and a fixed confidence threshold of 0.35, matching what the deployed browser app runs at, it reaches a raw mAP@0.5 of 82.28%, label-gap-corrected to 82.72%; precision 79.30%, recall 72.31%, F1 75.61%; corrected per-class AP of 86.39% for closed_eye, 83.00% for open_eye and 78.77% for yawning.

It was trained with AdamW (lr0 0.001 decaying to lrf 0.01 on a cosine schedule, momentum 0.9, weight decay 5e-4), a one-epoch warmup, box/cls/dfl loss weights of 7.5/1.5/1.5, batch size 64, for 25 epochs at 480px with a patience of 15, initialised from the 960px calibration checkpoint as a warm start. Augmentation used the most aggressive “worst-case” tier in the sweep: hue/saturation/value

jitter of 0.03/0.8/0.6, rotation up to 25°, translation 0.25, scale 0.7, shear 8°, perspective 0.0008, horizontal flip 0.5, random erasing 0.4 and RandAugment, while mosaic, mixup, cutmix and copy-paste compositing were disabled, as they were for every run in the sweep. The training checkpoint is 5.11 MB; the browser-deployed ONNX export is 9.28 MB in fp32 or 4.71 MB in fp16. Measured on an RTX 2000 Ada at batch size 1 over 50 iterations, GPU inference runs at 10.29 ms (97.2 FPS, p95 11.38 ms) and CPU inference at 49.75 ms (20.1 FPS).

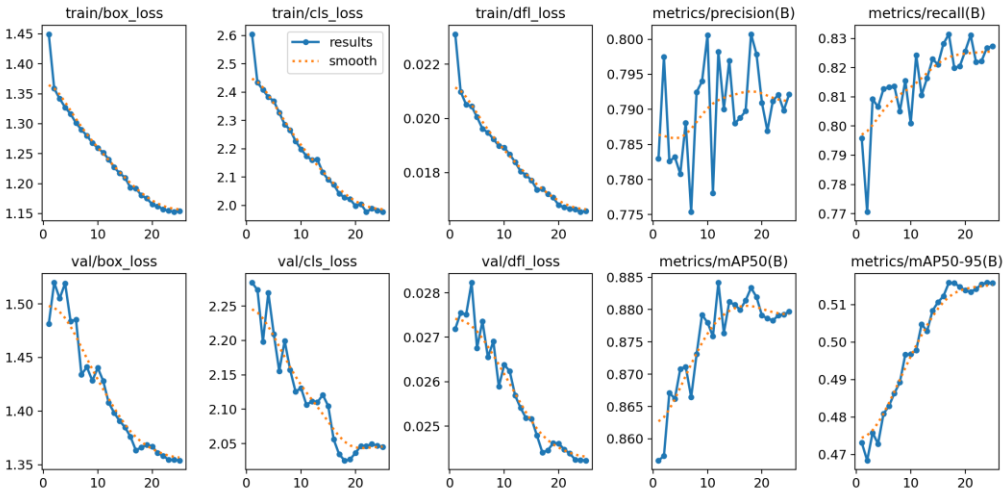

Fig. (A.1) YOLO26n 480px: training/validation loss and metric curves (25 epochs)

A.3.2 YOLO26n — 960px (Deployed, High Tier)

This is the high-tier deployed configuration, checkpointed at checkpoints/yolo26n/4-calibration-yolo26n-960-moderate-aug/best.pt with configuration at .../args.yaml. Under the same evaluation protocol as the 480px model, it reaches a raw mAP@0.5 of 82.34%, corrected to 82.75%; precision 78.99%, recall 73.37%, F1 76.06%; corrected per-class AP of 88.69% for closed_eye, 83.27% for open_eye and 76.28% for yawning.

It shares the same optimizer family as the 480px model, but with a three-epoch warmup, batch size 32, 40 epochs at 960px, a patience of 12, and initialisation from an earlier fine-tuning checkpoint. Augmentation used the noticeably gentler “moderate” tier: hsv 0.02/0.7/0.5, rotation 15°, translate 0.15, scale 0.6, shear 4°, perspective 0.0005, erasing 0.3, with the same compositing-disabled policy as every other run. The training checkpoint is 5.20 MB; the deployed ONNX export is 9.55 MB fp32 or 4.84 MB fp16. This run's own native Ultralytics chart images were never saved to disk - only its results.csv survives - so the figure below was regenerated from that retained log rather than reproduced from an original chart file.

The benchmarked latency for this checkpoint is unreliable and is flagged rather than quoted as final: the recorded pass read 21.58 ms / 46.4 FPS on GPU, roughly double every architecturally

identical sibling run, which was traced to measurement contamination from background system load during that specific benchmarking pass rather than a real property of the model. An informal re-measurement taken during a later autotune sweep gave 9.46 ms (≈106 FPS), consistent with the 480px model's pattern, but neither number should be treated as final without a clean, dedicated re-benchmark.

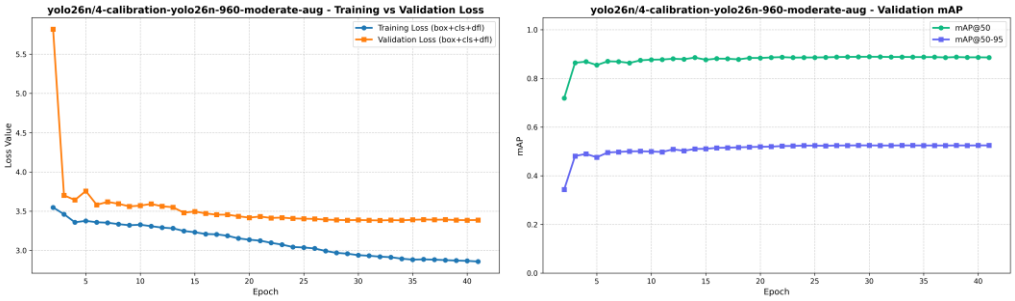

Fig. (A.2) YOLO26n 960px: training/validation loss and mAP curves, regenerated from the retained results.csv (41 epochs logged)

A.3.3 RF-DETR — Nano Fine-Tuned (Best of the RF-DETR Family)

Checkpointed at checkpoints/rfdetr-nano/old-2-finetune-384/checkpoint_best_ema.pth, exported to .../rfdetr-nano.onnx, with hyperparameters self-reported in .../info.md. This is the best-performing configuration among the five RF-DETR runs trained: nano fine-tune (78.99%) outperforms nano baseline (78.98%), small worst-case (73.55%), small standard (73.12%) and small baseline (66.21%), all corrected mAP@0.5 under this project's own evaluation protocol. Re-evaluated on the project's 5,589-image test set, it reaches a raw mAP@0.5 of 78.29%, corrected to 78.99%; precision 62.34%, recall 85.90%, F1 72.10%; corrected per-class AP of 97.69% for closed_eye, 82.02% for open_eye and 57.27% for yawning.

A second, conflicting number exists for this exact checkpoint: its own info.md reports an old-project self-evaluation of mAP50 92.36%, precision 75.62%, recall 94.61%. That figure is not used in this document - the old evaluation pipeline behind it was found to reuse byte-identical confusion-matrix and loss-curve images across eleven unrelated runs spanning five architectures, so its accompanying test metrics cannot be trusted either. The 78.99% figure above, measured under this project's own single consistent protocol on the real 5,589-image test set, is the one used throughout this document.

Hyperparameters (self-reported in info.md, since RF-DETR training does not produce an args.yaml): AdamW with a decoder learning rate of 2e-5 and encoder learning rate of 1e-6, weight decay 1e-4, box(giou)/cls loss weights of 5.0/1.5, 15+15 epochs, a real input size of 384px, and a total wall-clock time of 11.84 hours. Augmentation is documented only qualitatively, since no per-

parameter block exists for RF-DETR: extreme low-light/IR-glare simulation, brightness/contrast jitter of 0.35, and rotation up to ±15°. The checkpoint is 115.2 MB (.pth) and its ONNX export is 108.1 MB. Its parameter count, measured directly by loading the model and summing tensor elements rather than taken from info.md's own "≈10M" estimate, is 30.15M. Latency at batch size 1 is 11.29 ms (88.6 FPS, p95 15.41 ms) on GPU and 81.30 ms (12.3 FPS) on CPU; this benchmark row was not flagged as contaminated.

No per-epoch training log was initially thought to survive for any RF-DETR run, since the checkpoint itself stores only a final epoch counter. A genuine TensorBoard event log for this exact run was later located under a parent folder one level different from the path recorded in the checkpoint's own metadata, confirmed to match by step count: 15 logged points at a spacing of roughly 2,086 steps, reaching global_step 31,290 - exactly what the checkpoint itself records at 15 epochs. It shows training loss falling from 5.396 to 4.734 and validation mAP@0.5 holding in a narrow 91.7-92.0% band throughout, indicating this run was fine-tuning an already-strong checkpoint, so most of the learning happened before this log begins rather than during it. This is training-time validation on the old project's own split, not this project's 5,589-image held-out test set, and does not change the 78.99% figure used for comparison.

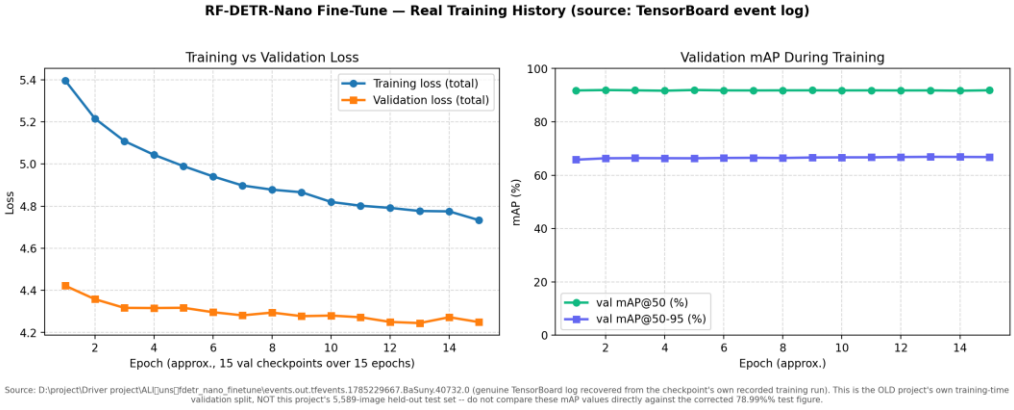

Fig. (A.3) RF-DETR-Nano fine-tune: recovered real per-epoch training/validation loss and mAP curves, from the checkpoint's own TensorBoard log (15 epochs)

A.3.4 YOLO11m — Warm-Start (Highest Overall Accuracy)

Checkpointed at checkpoints/yolo11m/1-yolo11m-warmstart-pilot-640/best.pt with configuration at .../args.yaml. This is the single most accurate model trained across all 21 runs, at 86.75% corrected mAP@0.5 - ahead of the next-best YOLO11n-capacity run (83.11%) and every YOLO26n variant - though it was never exported or deployed. On the project's 5,589-image test set it reaches a raw mAP@0.5 of 86.42%, corrected to 86.75%; precision 82.69%, recall 73.82%, F1

77.97%; corrected per-class AP of 92.62% for closed_eye, 87.01% for open_eye and 80.61% for yawning.

It was trained with AdamW (lr0 0.001 to 0.01 on a cosine schedule), batch size 16, for 15 epochs at a patience of 15 - running the full training budget without triggering early stopping - at 640px, with a one-epoch warmup and box/cls/dfl loss weights of 7.5/1.5/1.5. It was initialised from a cross-dataset YOLO11m checkpoint produced by a separate prior run, a warm start that let it reach this accuracy in only 15 epochs. Augmentation used the same "moderate" tier as the 960px YOLO26n model above. The training checkpoint is 38.64 MB; this family produced no browser ONNX or fp16 export, since it was never selected for deployment - at 20.06M measured parameters it is a substantially heavier browser download than either deployed YOLO26n model for a roughly four-point accuracy gain, discussed further in the comparison below. Latency at batch size 1 is 11.26 ms (88.8 FPS, p95 13.42 ms) on GPU and 151.67 ms (6.59 FPS) on CPU.

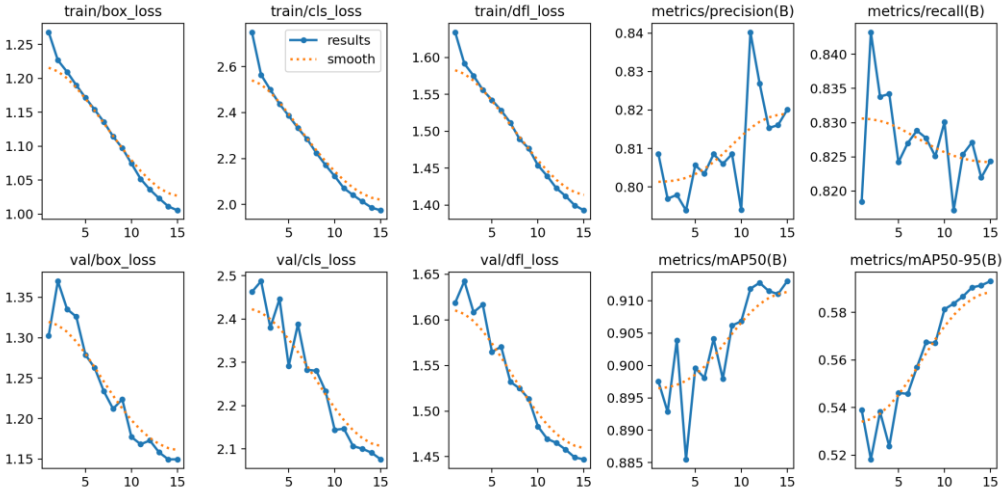

Fig. (A.4) YOLO11m warm-start: training/validation loss and metric curves (15 epochs)

A.3.5 From-Scratch Faster R-CNN — Tuned Configuration

This is the same tuned checkpoint discussed in Section 3.4.4 and Section 4.8 (Table 4.5), checkpointed at D:/project/Driver project/michel from scartch/checkpoints/tuned/best.pth; it is repeated here with its full hyperparameter and evidence record for consistency with the other four models in this appendix. It is not directly comparable to the four models above, since it was evaluated on a differently sized 5,705-image test set rather than this project's own 5,589-image set. Tuning improved its test metrics over the baseline configuration: mAP@0.5 from 72.61% to 74.27%; mAP@0.5:0.95 from 32.76% to 34.60% - the only model in this comparison with this stricter metric recorded; precision from 70.62% to 71.02%, recall from 82.47% to 82.60%, F1 from 76.09% to

76.37%; per-class AP from 74.65% to 73.15% for closed_eye, 64.70% to 67.57% for open_eye, and 78.47% to 82.10% for yawn.

Hyperparameters (a custom implementation with no framework defaults): SGD with a learning rate of 0.005, momentum 0.9, weight decay 5e-4, batch size 4, 50 epochs, 640px, ReduceLROnPlateau (factor 0.5, patience 3), and an early-stop patience of 8. Sixteen anchors per cell (4 scales × 4 ratios) were used, with RPN positive/negative IoU thresholds of 0.7/0.3 and an RoI foreground IoU of 0.5. Augmentation combined horizontal flip (0.5), colour jitter (brightness/contrast 0.3, saturation 0.2, hue 0.02) and an affine transform (rotation up to 8°, scale 0.8-1.2, translation 0.10), each applied with 0.5 probability. The checkpoint is 128.6 MB and its measured parameter count, read directly from the state dictionary, is 16.86M. No latency benchmark exists for this model - it was never run through the benchmarking script used for the other four. The further refined, 65-epoch extension of this run, reaching mAP@0.5 77.84%, is described separately in Section 4.8 and Table (4.5) and is not repeated here.

The 16.86M figure above was re-verified by a fresh reload of the checkpoint (checkpoints/tuned/best.pth, epoch 50): the exact total is 16,858,028 parameters, of which 2,824 are non-trainable BatchNorm buffers (running_mean/running_var), leaving 16,855,204 trainable weights - a negligible difference. The breakdown by component is:

Component	**Params**	**Share**
ROI head (classifier + box regressor)	13,916,180	82.5%
Backbone (custom CNN, no ImageNet pretraining - hand-built in models/backbone.py, BackboneCNN class, not a torchvision ResNet)	2,331,208	13.8%
RPN	610,640	3.6%
Total	16,858,028	100%

Table (A.1) Faster R-CNN (from-scratch, tuned) parameter count by component

The ROI head dominates the model at 82.5% of all parameters, consistent with a Faster R-CNN built on a small, custom backbone rather than a heavy pretrained feature extractor: in a ResNet-based Faster R-CNN the backbone itself would typically be the largest component, whereas here the hand-built backbone (models/backbone.py, BackboneCNN class) accounts for only 13.8% of parameters. For scale, this places the model in roughly the same order of magnitude as RF-DETR-Nano (30.15M) but smaller, sitting between YOLO11m (20.06M) and YOLO26n (2.50M) in Table (A.1) below.

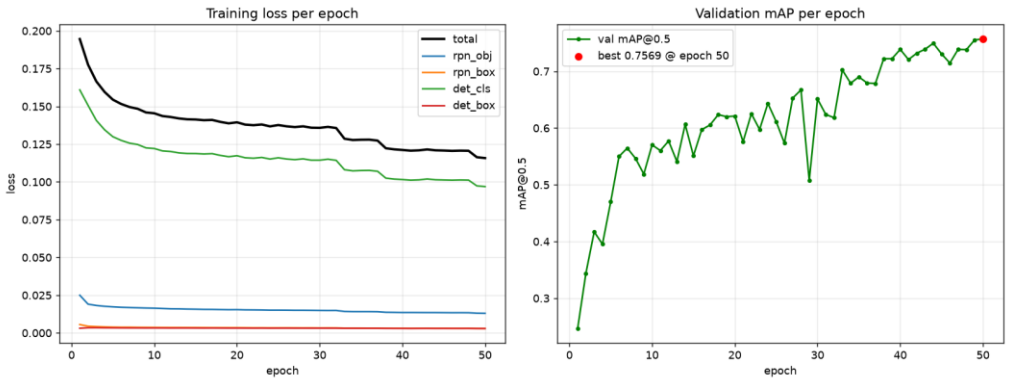

Fig. (A.5) From-scratch Faster R-CNN, tuned configuration: training loss (four components) and validation mAP@0.5 across 50 epochs

Table (R.1) Reference comparison: model used, data accessibility, and principal limitation

#	Model / Approach Used	Ease of Obtaining Their Data	Biggest Problem / Limitation
[1]	Semi-automated, human-in-the-loop labelling pipeline producing frame-level yawn annotations for YawDD (no detection model of their own).	Re-annotations released online at a public repository; but single-task (yawning only) frame labels layered on one source video set.	The dataset they start from carries systematic noise from coarse, video-segment-level temporal annotation.
[2]	Lightweight YOLOv5s face/eye detector combined with 3D facial keypoints; eye/mouth state inferred from aspect-ratio thresholds, not learned directly.	Only the four source datasets (YawDD, CEW, DrivFace, DROZY) are individually linked; the authors' own compiled 8,021-image working set has no separate release.	Landmark-based geometric ratios depend on reliable landmark localisation, which degrades under poor illumination, occlusion, and extreme pose.
[3]	MG-YOLOv8: a deliberately lightweight, real-time detector combining multiple facial features for fatigue detection.	The two benchmark datasets (WIDER FACE, YawDD) are public; the paper's own 12-participant in-house supplementary data is not released.	Explicitly trades some accuracy for a latency budget realistic on target hardware, so headline accuracy is not directly comparable to heavier models.
[4]	Benchmarks seven face detectors (YOLOv11n, SSD MobileNet, CenterFace, YuNet, FastMTCNN, HaarCascade, LBP) rather than proposing a single new one.	No availability statement is given for either the source videos (NITYMED) or the authors' own frame-level annotations (~1,229 frames).	Notes that many prior studies use detector-generated outputs as their own ground truth, which can bias and inflate reported performance.
[5]	Fine-tunes seven YOLO variants spanning v5 through v11 on UTA-RLDD and compares them head-to-head.	Called “publicly available” in the paper, but the source dataset (identifiable face video) has historically required a request/consent process with the originating researchers.	The highest-accuracy variant (YOLOv9c) and the best-balanced variant for embedded deployment (YOLOv11n) are not the same model - accuracy and deployability diverge.
[6]	Bayesian-optimised CLAHE contrast enhancement feeding a time-distributed MobileNetV2-GRU classifier, targeting low-light frames specifically.	No availability statement is given for the NITYMED dataset used.	Addresses image quality (illumination) rather than label quality; adds inference-time preprocessing cost that this project's photometric-augmentation approach avoids.
[7]	Multi-level feature fusion with a long short-term recurrent network (LSTM), treating drowsiness as a temporal pattern rather than a per-frame property.	No availability statement is given for the dataset used (NTHU-DDD, obtained “with the permission of” the source laboratory - permission-gated, not an open download).	Temporal modelling is identified as future work for the present project rather than something adopted here; per-frame detection is the scope of this thesis.
[8]	None - a literature survey identifying deployment obstacles across the ADAS drowsiness-detection field; proposes no model or dataset of its own.	Not applicable (survey paper, no dataset).	Being a survey, it does not report an empirical benchmark of its own to compare against; its contribution here is the deployment-obstacle framing used in Section 2.1.

References

[1] A. Mujtaba, G. Radchenko, M. Masana, and R. Prodan, "YawDD+: Frame-level annotations for accurate yawn recognition on edge platforms," Silicon Austria Labs, Graz University of Technology, and University of Innsbruck, 2025.

[2] M. Arava and D. M. Sundaram, "Integrating lightweight YOLOv5s and facial 3D keypoints for enhanced fatigued-driving detection," PeerJ Computer Science, vol. 10, e2447, 2024.

[3] C. Chen, X. Liu, M. Zhou, Z. Li, Z. Du, and Y. Lin, "Lightweight and real-time driver fatigue detection based on MG-YOLOv8 with facial multi-feature fusion," Journal of Imaging, vol. 11, no. 11, p. 385, 2025.

[4] A. A. D. Go, F. Alzami, M. Naufal, H. Al Azies, S. Winarno, R. A. Pramunendar, R. A. Megantara, I. I. Maulana, and M. Arif, "Comprehensive benchmark of YOLOv11n, SSD MobileNet, CenterFace, YuNet, FastMTCNN, HaarCascade, and LBP for face detection in video based driver drowsiness," Building of Informatics, Technology and Science (BITS), vol. 7, no. 3, pp. 1775-1784, 2025.

[5] D. Herath, C. Abeyrathne, and P. Jayaweera, "Vision-based driver drowsiness monitoring: Comparative analysis of YOLOv5-v11 models," University of Ruhuna, Sri Lanka, 2025.

[6] F. Alzami, M. Naufal, R. S. Basuki, S. Winarno, H. Al Azies, S. L. Lutfi, and R. M. Brilianto, "Bayesian-optimized CLAHE for enhanced drowsiness detection in low-light conditions using time-distributed MobileNetV2-GRU architecture," Statistics, Optimization and Information Computing, vol. 151, pp. 274-294, 2026.

[7] L. Yusuf, M. Hamada, M. Hassan, and H. Kakudi, "Enhanced driver drowsiness detection model using multi-level features fusion and a long-short-term recurrent neural network," Engineering Proceedings, vol. 56, no. 1, p. 338, 2024.

[8] H. George and L. Rochit, "Advancing driver assistance systems and drowsiness detection: Overcoming challenges for enhanced road safety," TechRxiv preprint, 2025.

Table (R.1) below organises the eight reviewed studies by what they actually did, how accessible their underlying data is, and the principal limitation of each - summarising, in one place, the comparison that motivates the data-collection and evaluation choices made in Chapter 3.

ملخص المشروع

يمثل إرهاق السائق أحد الأسباب المستمرة لحوادث الطرق، ويمكن ملاحظة مؤشراته السلوكية المبكرة — إغلاق العينين لفترات طويلة والتثاؤب — من خلال كاميرا واحدة داخل المركبة. يقدم هذا المشروع نظامًا متكاملًا لمراقبة السائق يعتمد على الرؤية الحاسوبية، بدءًا من بناء مجموعة البيانات ومرورًا باختيار النموذج وانتهاءً بالنشر داخل المتصفح.

بدأ العمل من مجموعة بيانات تضم 57,098 صورة جُمعت من عدة مصادر مُوسمة بشكل مستقل. أظهر التحليل أن هذه المجموعة كانت دمجًا لمجموعات بيانات أحادية المهمة — مجموعة لحالة العين، ومجموعة للتثاؤب، وتسجيلات لجلسات مراقبة السائق — دُمجت في فضاء تسميات ثلاثي الفئات دون إعادة توسيم. نتج عن ذلك نقص منهجي في الإشراف مرتبط بالمصدر وليس ضوضاء عشوائية في التسميات. وقد أنتجت عملية مراجعة منظمة، شملت تحققًا بشريًا من عينات من الصور، سجلًا للإشراف يراعي المصدر. كما طُبق تقسيم واعٍ بالمجموعات لمنع تسرب الإطارات المتشابهة بين مجموعتي التدريب والاختبار. تحتوي مجموعة البيانات النهائية على 50,654 صورة و68,292 حالة مُوسمة عبر الفئات: العين المغلقة، والعين المفتوحة، والتثاؤب.

أُجريت إحدى وعشرون تجربة تدريب عبر أربع عائلات معمارية للكشف عن الأجسام — YOLO26 وYOLOv11 وRF-DETR القائم على المحوّلات، بالإضافة إلى نموذج Faster R-CNN مبسّط مُنفّذ من الصفر — بإجمالي 950 حقبة تدريبية و277.57 ساعة معالجة رسومية، وبدقات إدخال تتراوح بين 384 و960 بكسل. وقد قُيّمت جميع النماذج وفق بروتوكول موحّد على مجموعة اختبار مستقلة تضم 5,589 صورة.

تم اختيار نموذجين من YOLO26n للنشر، بدقة إدخال 480 و960 بكسل، وحققا 82.72٪ و82.75٪ على التوالي وفق مقياس mAP@0.5 المصحّح. وقد صُدّر كلا النموذجين إلى صيغة ONNX مع دمج خطوة الكبت غير الأقصى داخل الرسم البياني، وتحويلهما إلى دقة نصفية، مما خفّض حجم النموذج إلى النصف دون خسارة تُذكر في دقة الكشف. ويجري تنفيذ النماذج بالكامل على جهاز المستخدم عبر تطبيق يعمل في المتصفح، بحيث لا يُنقل الفيديو إلى أي خادم خارجي.

يتم التفسير الزمني عبر تجميع PERCLOS ضمن نافذة متحركة مع آلة حالة مُثبّتة. أما النمذجة الزمنية المُتعلّمة باستخدام المعماريات التكرارية أو التسلسلية فلم تُنفّذ في هذا العمل، وقد حُدّدت كمرحلة تطوير لاحقة.

الجهة المانحة

صفحة الموافقة

نظام قائم على الذكاء الاصطناعي لسلامة السائق والمساعدة أثناء القيادة

رسالة مقدمه من

محمد مصطفى محمد البسيوني (قائد الفريق)

مايكل مجدي أمين سدهم

علي إبراهيم أحمد عثمان

كريم مصطفى علي إبراهيم

محمد أسامة بهنسي عبد الحليم

رسالة مقدمة استكمالًا لمتطلبات الحصول على

دبلومة Digilians 9 Months
مسار الذكاء الاصطناعي التطبيقي وتحليل البيانات

يعتمد من لجنة الممتحنين

الأسم	التوقيع
أ.د /	
أ.د /	
أ.د /	

الجهة المانحة

نظام قائم على الذكاء الاصطناعي لسلامة السائق والمساعدة أثناء القيادة

رسالة مقدمة من

محمد مصطفى محمد البسيوني (قائد الفريق)

مايكل مجدي أمين سدهم

علي إبراهيم أحمد عثمان

كريم مصطفى علي إبراهيم

محمد أسامة بهنسي عبد الحليم

تحت إشراف

لواء ا. د./ كامل الحداد

كليه الفنية العسكرية

رسالة مقدمة استكمالًا لمتطلبات الحصول على دبلومة Digilians لمدة تسعة أشهر في الذكاء الاصطناعي التطبيقي وتحليل البيانات

القاهرة 2026